

# 华中科技大学

## 《软件工程》项目报告

题目： 喻园易站

课程名称： 软件工程

专业班级： CS23

组 名： 喻园易站小组

同组成员： 学号： U2023

姓名：

学号： U2023

姓名：

学号： U2021

姓名：

学号： U2023

姓名：

指导教师： 范晔斌

报告日期： 2025-12-1

计算机科学与技术学院

## 任 务 书

### 一 总体要求

1. 综合运用软件工程的思想，协同完成一个软件项目的开发，掌软件工程相关的技术和方法；
2. 组成小组进行选题，通过调研完成项目的需求分析，并详细说明小组成员的分工、项目的时间管理等方面。
3. 根据需求分析进行总体设计、详细设计、编码与测试等。

### 二 基本内容

根据给出的题目任选一题，自行组队，设计与开发中软件过程必须包括：

1. **问题概述、需求分析：**正确使用相关工具和方法说明所开发软件的问题定义和需求分析，比如 NABCD 模型，Microsoft Visio，StarUML 等工具 (20%)；
2. **原型系统设计、概要设计、详细设计：**主要说明所开发软件的架构、数据结构及主要算法设计，比如墨刀等工具（35%）；
3. **编码与测试：**编码规范，运用码云等平台进行版本管理，设计测试计划和测试用例（30%）；
4. **功能创新：**与众不同、特别吸引用户的创新（10%）；
5. **用户反馈：**包括用户的使用记录，照片，视频等（5%）。

## 目录

|                      |    |
|----------------------|----|
| 任务书.....             | 1  |
| 1 问题定义.....          | 3  |
| 1.1 项目背景与意义.....     | 3  |
| 1.2 项目基本目标.....      | 4  |
| 1.3 可行性分析.....       | 4  |
| 1.4 人员管理和项目进度管理..... | 5  |
| 2 需求分析.....          | 8  |
| 2.1 需求分析概述.....      | 8  |
| 2.2 UML 相关需求分析图..... | 9  |
| 2.3 原型系统设计.....      | 12 |
| 3 概要设计和详细设计.....     | 14 |
| 3.1 系统结构.....        | 14 |
| 3.2 设计视图.....        | 15 |
| 3.3 关键数据结构定义.....    | 16 |
| 3.4 关键算法设计.....      | 23 |
| 3.5 数据管理说明.....      | 24 |
| 4 实现与测试.....         | 30 |
| 4.1 实现环境与代码管理.....   | 30 |
| 4.2 关键函数说明.....      | 32 |
| 4.3 测试计划和测试用例.....   | 35 |
| 4.4 结果分析.....        | 41 |
| 5 总结.....            | 43 |
| 5.1 用户反馈.....        | 43 |
| 5.2 全文总结.....        | 44 |
| 6 体会.....            | 46 |
| 附录.....              | 49 |

# 1 问题定义

## 1.1 项目背景与意义

### 1.1.1 项目背景

在现代校园中，师生除了学习和工作，日常生活中的物品交换、信息求助、失物招领等方面的需求也日益增加。然而，目前这些需求在校园内部的解决方式并不高效。闲置物品流通不畅、互助求助信息分散以及失物招领流程繁琐，已经成为师生日常生活中的常见痛点。每年，许多学生在毕业季或换季时丢弃大量的闲置物品，这不仅造成了资源的浪费，也降低了物品的再利用率。而信息求助的渠道多为碎片化的小范围群聊，解决问题的效率低下，很多问题因信息的传播不及时而难以得到有效解决。另一方面，校园内的失物招领机制也存在较大的缺陷，不同的渠道之间缺乏有效的信息同步，拾到物品和寻找失物的过程非常低效，严重影响了师生的体验。

为了应对这些问题，我们决定构建一个综合性的平台，集**闲置物品交易**、**互助问答**、**跑腿代办**和**失物招领**为一体，旨在通过高效的信息流通和资源利用，提升校园生活的便捷性和效率。这不仅可以帮助学生更好地处理闲置物品，提供更快捷的求助途径，还能大幅提高失物找回的概率，改善整体校园环境。

### 1.1.2 项目意义

“喻园易站”项目的意义在于它能够为校园师生提供一个集交易、问答与失物招领为一体的高效平台，极大地方便了师生的日常生活。平台将有效打破校园内的信息孤岛，提供一个全校范围的信息整合系统，提升资源的共享与交换效率。学生不仅能够更方便地出售和购买二手物品，还可以在日常生活中遇到问题时，通过平台获得及时的帮助和支持。

同时，平台的推出也为校园内的社区互动提供了新的渠道。通过“帮帮忙”和“失物招领”两个功能，学生能够在平台上主动寻求帮助，也能够为他人提供帮助，从而营造出更加和谐的校园氛围。对于学校而言，平台能够提升校园信息的流动性和透明度，减少资源浪费，鼓励循环经济，为学校的可持续发展贡献力量。

量。

在经济层面，平台不仅有助于学生在低成本的环境下获取所需物品，还为学生提供了便捷的物品变现渠道，帮助他们减轻经济负担。此外，平台的运行还能够为校园内的小型商家提供更为便捷的线上交易通道，推动校园经济的良性循环。

## 1.2 项目基本目标

“喻园易站”项目的基本目标是构建一个以校园为核心的综合性服务平台，主要功能包括闲置物品交易、互助问答、跑腿代办和失物招领。通过该平台，学生能够在校园内更便捷地进行物品交易，分享资源并减少浪费，同时也可以通过平台快速获得帮助，解决在学习或生活中遇到的问题。

具体而言，平台将提供一个便捷的物品发布与搜索功能，支持图片上传和商品分类，帮助学生轻松进行物品交易。在互助问答模块中，平台将提供一个集中化的信息发布渠道，允许用户发布问题并获得其他用户的回复，从而解决学习和生活中的各种困难。失物招领功能则通过整合全校的失物信息，为丢失物品的师生提供一个高效、统一的寻物平台，提升物品的找回率。图 1-1 展现了项目的基本目标的功能。

此外，项目还将考虑未来的扩展性，逐步引入更多功能，如用户信用体系、在线支付系统等，为师生提供更加完善的服务，提升平台的用户粘性。

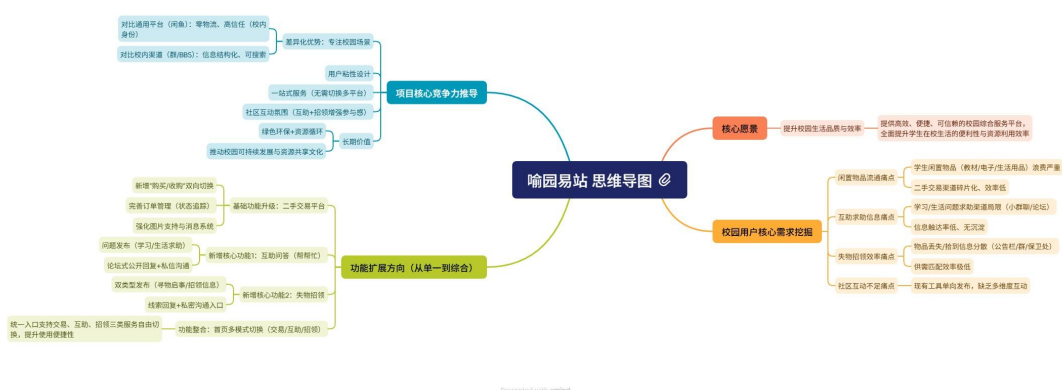


图 1-1 项目思维导图

## 1.3 可行性分析

从技术角度来看，本项目的可行性较高。我们选择了现代化的技术栈来开发该平台，前端采用 React 与 Vite 框架，结合 Tailwind CSS，实现响应式和高性能的单页面应用（SPA），使得平台界面更加流畅，用户体验得到保障。后端使用

Node.js 与 Express 框架，基于 RESTful API 提供高效的数据服务，确保平台能够在高并发的情况下平稳运行。同时，平台的数据库采用 MySQL，确保数据的稳定性与可扩展性，满足未来功能扩展的需求。为了提升用户体验，平台还将引入 WebSocket 技术来支持即时通信功能，确保交易和互动的实时性。

在经济可行性方面，项目具备显著的市场价值。平台的运营可以通过一定的广告收入、增值服务（如优先展示、推广等）以及潜在的商家合作来获得收益。对于学生来说，平台的闲置物品交易将极大减少物品流通的成本，同时，平台的低成本运营模式也使其具备较强的市场竞争力。平台的运作将促进校园内闲置资源的高效利用，不仅提高了经济效益，也在一定程度上推动了校园内绿色环保理念的落实。

从社会角度来看，项目也具备较高的可行性。平台的建设有助于提升校园师生之间的互动交流和互助氛围。通过“帮帮忙”和“失物招领”模块，平台不仅解决了实际的生活问题，还为学生创造了一个更为和谐、团结的社会环境。该平台的推出能够有效提升学生对信息获取的信任度，增强校园社区的凝聚力和参与感，从而形成更为紧密的校园生态。

## 1.4 人员管理和项目进度管理

### 1.4.1 人员管理

“喻园易站”项目团队由四名核心成员组成，分别承担不同的技术与管理职责，以实现项目的高效协同与阶段性目标的稳步推进。根据《项目进度计划》文件显示，团队成员包括小A、小B、小C与小D，各自职责明确，分工合理，形成了一个结构紧凑、功能互补的开发小组。

其中，小A担任**项目负责人和系统架构师**，负责主要文档和报告撰写、项目部署等项目全过程。全面负责系统架构设计与技术路线规划，并主导需求分析、前后端框架的搭建、核心认证模块的实现以及团队技术协调工作。是项目的技术核心，负责保障整体代码架构的可扩展性与一致性，负责管理码云平台仓库，检查成员代码。

小B主要负责**“闲置交易”模块的优化和 Debug** 工作，包括帖子发布、订单系统及后续互助、招领模块的扩展开发。在项目开发周期中，他主导了部分与用

户交互相关的业务逻辑与数据库设计任务。

小C负责辅助用户中心与社交功能的开发，负责实现用户私信、好友关系、资料管理等社交模块，确保平台具备较强的用户黏性与互动性。其任务覆盖了前端界面与 WebSocket 通信后端的实现，是系统用户体验优化的关键角色。并和小A负责最后一节课的演示工作。

小D主要负责项目后期的测试工作。

这种以模块划分责任的组织结构在软件工程项目中较为高效，既能保证各开发阶段的技术深度，又利于成员间并行协作。通过定期会议与版本迭代评审，团队在进度管理上能够动态调整开发优先级，保持项目整体目标与进展的统一。

#### 1.4.2 项目进度管理

项目的总体开发周期为七周，从 2025 年 9 月 16 日启动至 2025 年 11 月 4 日正式上线。项目共划分为六个主要阶段，每个阶段均设定了清晰的任务目标与责

在阶段零：需求分析与设计期间（9 月 9 日至 9 月 15 日），团队集中进行项目目标与范围的确定、用户需求收集及系统概要设计。这一阶段主要由小A负责，为后续开发奠定了坚实的技术与逻辑基础。

阶段一：项目初始化与核心后端开发（9 月 16 日至 9 月 22 日）重点完成后端框架（Express）的搭建、数据库初始化及用户认证 API 的开发，实现了系统的基本数据流转功能。在此基础上，后续又利用 Nodemail 实现了校园邮箱认证机制，这一阶段主要由小A负责。

阶段二：前端基础与用户认证（9 月 22 日至 9 月 29 日）集中在前端架构搭建与 UI 组件设计上，采用 Vite+React 框架实现前端视图结构，并完成了注册与登录的前后端联调。

阶段三：核心功能——闲置交易模块（9 月 27 日至 10 月 13 日）是开发周期中最为关键的一阶段，小B负责完成了帖子发布、订单管理、交易状态流转等功能，实现了平台的业务闭环。

阶段四：核心功能——互助与招领模块（10 月 14 日至 10 月 20 日）在前期架构基础上扩展了数据模型，新增了论坛式回复功能、评论点赞与多模式切换界面，为平台增加了强社交属性。

阶段五：用户中心与消息模块（10月21日至10月27日）由小C主导，开发了个人中心页面、消息私信系统与好友机制，进一步完善了用户生态。

阶段六：测试、部署与收尾（10月28日至11月4日）由小D负责完成，包括系统集成测试、Bug修复、部署上线及撰写项目总结报告，标志着项目正式进入交付阶段。

在进度控制方面，团队采用了甘特图式进度可视化管理，将各阶段任务的起止时间、负责人及完成比例以图形化方式呈现，如图 1-2 所示。这一方式帮助团队成员清晰掌握项目进展，方便在开发中期进行风险预判与资源协调。通过每周例会与阶段性评审，团队能够有效识别开发瓶颈并及时调整任务优先级，从而保持整体进度与预定计划的一致性。

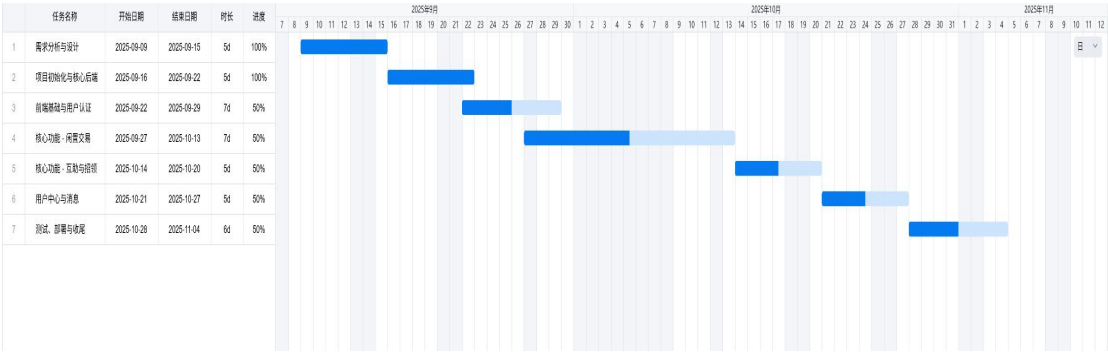


图 1-2 甘特图

## 2 需求分析

### 2.1 需求分析概述

在开展本项目之前，团队通过调研发现校园环境中师生在物品交易、互助问答、跑腿代办与失物招领四个方面存在显著需求缺口。首先，在闲置物品流通方面，毕业季或换季时许多师生手中拥有大量未使用的教材、电子产品或生活用品，但缺乏一个便捷快速、统一、可信赖、校内专属的交易平台，这造成资源浪费和重复采购。其次，在生活或学习过程中，师生遇到疑问或困难（例如课程资料、学习方法、生活服务设施），当前多数通过微信群、QQ 群或线下信息渠道求助，这种方式信息扩散慢、存留难、沉淀差。再次，在失物招领方面，不少物品丢失或被拾但难以归还，相关信息散落于多个渠道（如保卫处公告、微信群、校园微信小程序等），匹配效率低。

基于以上背景，系统明确了要实现的目标：提供一个“校内专属、功能综合、流程精简、用户信任度高”的服务平台。该平台将集成“闲置交易”、“互助问答”、“跑腿代办”、“失物招领”四大核心模块，使校园师生能够在一个入口完成发布、搜索、交流、匹配与反馈全过程。通过结构化的功能模块、在线化交互界面与后台管理机制，提升资源利用率、信息流通效率与社区互动质量。

为了直观展示用户需求与系统目标之间的对应关系，下面采用需求-目标映射表格予以说明：

表 2-1 需求-目标映射表

| 用户需求            | 系统目标                      |
|-----------------|---------------------------|
| 校园闲置物品流通困难      | 实现校内二手交易模块：发布、浏览、交易全流程    |
| 学习/生活中求助渠道分散、耗时 | 建立互助问答模块：集中发布、讨论、私信沟通     |
| 丢失/招领物品匹配效率低    | 提供统一失物招领模块：信息发布、线索反馈、私聊联系 |
| 多平台切换、信任度低      | 企业邮箱身份认证、统一平台入口、功能融合一体化   |

## 2.2 UML 相关需求分析图

### 2.2.1 用例图

在这张用例图中，首先明确了两个主要角色：学生 / 教职工作为平台的主体使用者，他们拥有登录、发布、浏览、私信、管理订单等功能；而管理员则承担审核与内容管理职责。图中“交易模块”“互助问答模块”“失物招领模块”三大功能板块，通过“发布、浏览、提交、私信”等用例体现出系统所支持的用户动作。可以看出，用例图很好地反映了系统需求中“用户可以切换模式（购买、收购、帮忙、失物招领）”这一特性，也体现了“实时消息”“多图片上传”“订单状态跟踪”等功能需求。

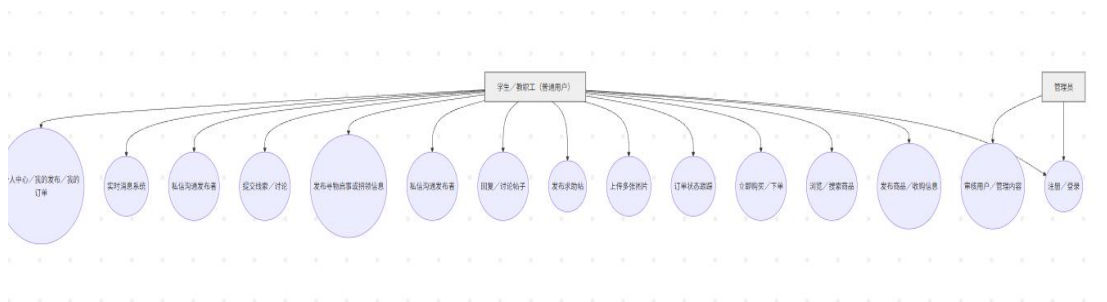


图 2-1 用例图

### 2.2.2 领域类图

该类图反映了系统中的核心实体及其关系。User 类是基础实体，代表平台注册用户；Post 类则用于统一描述三大功能模块中“帖子”这一共同结构（交易帖、求助帖、招领帖）——通过 type 区分不同模式，通过 status 描述状态变化。Image 支持多图上传，Category 则用于标签或分类功能。Order 是交易模块中的关键实体，其状态枚举体现了“待付款 / 待收货 / 已完成 / 取消”的流程。Message 体现了实时私信功能，Comment 则用于帖子中的互动回复与点赞。关联关系中表现出一名用户可以发布多个帖子、下多个订单，也可发送 / 接收多条消息，帖子与图片、评论、标签之间的关系也明确地被表示出来。由此，本类图将系统的数据模型结构以清晰、统一的方式展现出来，为后续数据库设计与开发提供了直接参考。

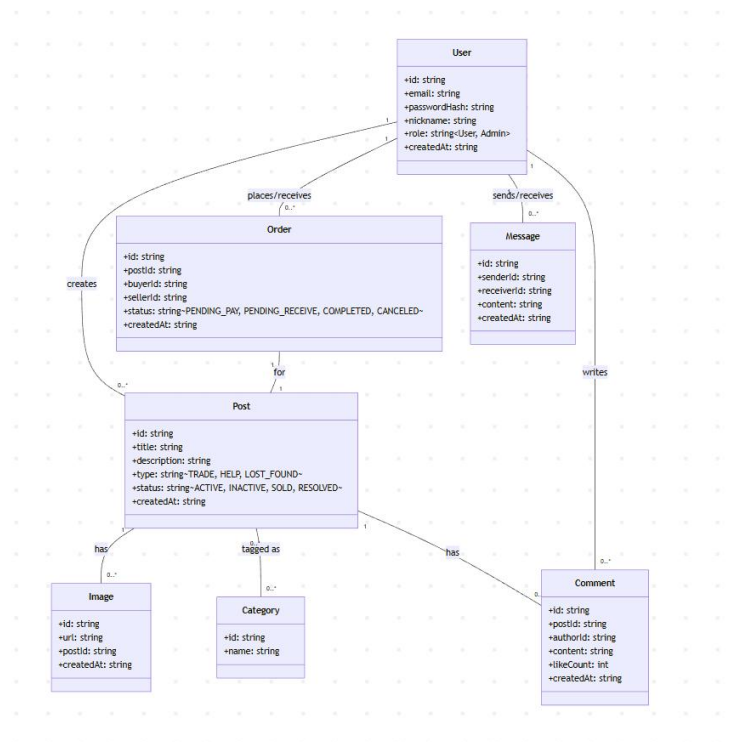


图 2-2 领域类图

### 2.2.3 下单序列图

这一序列图展示的是交易模块中买家下单、与卖家私信沟通、订单状态推进直至完成的关键业务流程。开始于用户选择商品、系统检索数据、建立订单，经过消息沟通，直到买家确认收货，流程结束。该图强调前端、后端、数据库及实时消息服务之间的协作。它契合文档中“立即购买”“订单状态管理”“消息系统”的描述，并为开发团队在构建各组件时提供了行为指南。

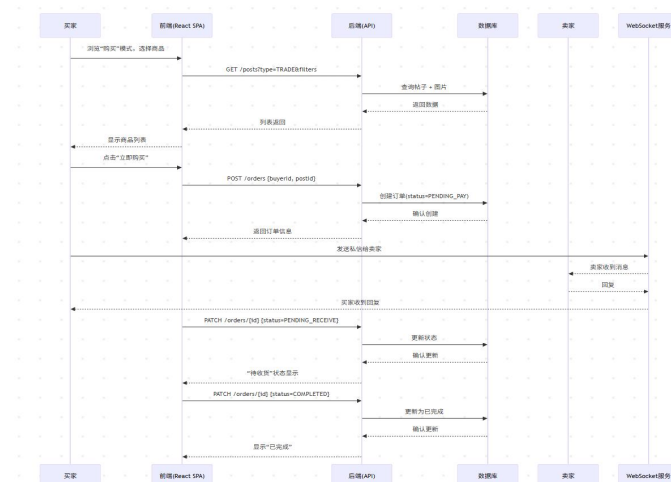


图 2-3 下单序列图

## 2.2.4 订单状态图

在这张图中，我们以状态机形式描绘订单从创建到完成或取消的生命周期。最初状态是 PENDING\_PAY（买家已下单但未付款）。若买家付款或卖家确认，进入 PENDING\_RECEIVE（待收货）阶段；待买家确认收货，转入 COMPLETED（已完成）。如果付款超时、买家主动取消，或卖家因故取消/退款，则分别进入 CANCELED（已取消）状态。该模型明确指出平台应支持的业务边界，也提醒开发时要考虑“超时取消”“退款处理”等异常流程。它与项目需求中“订单状态（待付款、待收货、已完成）管理”高度一致。

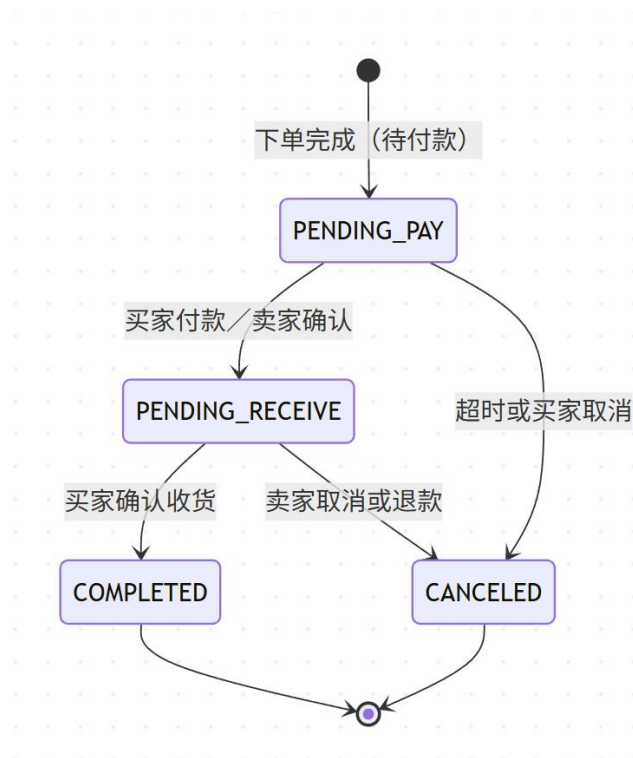


图 2-4 订单状态图

通过这四类 UML 模型，系统需求在不同维度被清晰地表达。用例图说明了谁做什么；类图揭示了系统中“是什么”；序列图强调了关键流程的“如何做”；状态图则明确了事务的生命周期如何变化。综合来看，这些模型不仅增强了需求分析的可检验性，也为接下来原型设计、系统架构、开发实施及测试提供了逻辑支撑。

## 2.3 原型系统设计

原型系统设计阶段通过 html 静态网页形式直观展现系统主要功能及用户交互界面，从而使需求分析更加具象化、可检验。

原型网站链接：<https://ilosyi.github.io/>可用于用户预览界面流转与交互逻辑，如图 2-1 所示

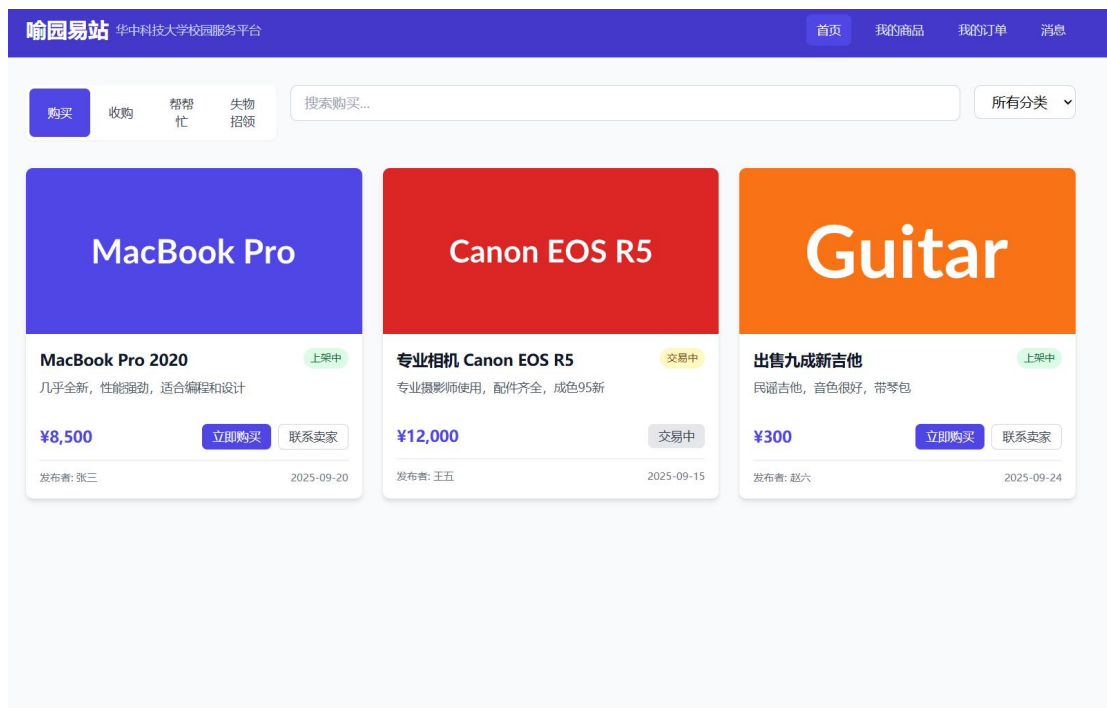


图 2-5 原型网站首页

在原型中，首页入口提供了四大模式切换：首页显示“购买”“收购”“帮帮忙”“失物招领”四种功能模块，用户可在顶部导航栏或主页卡片上快速切换。用户点击“购买”进入商品列表页面，支持图片预览、筛选排序、搜索功能；点击某一商品可进入详情页，包括商品描述、卖家信息、留言入口、立即购买按钮。订单状态管理（如待付款、待收货、已完成）也在“我的订单”页有所体现。该流程源码在仓库中 `pages` 目录下明确列出（例如 `MyOrdersPage.jsx`）。

在“帮帮忙”板块，用户可发布求助帖，帖内可附加问题描述、图片、类别（学习、生活、设施等）；其他用户可查看、回复、点赞，并进入私信系统与发布者沟通。私信聊天模块基于 `WebSocket` 实现即时通信，在原型中已通过页面 `MyMessagesPage.jsx` 展示。

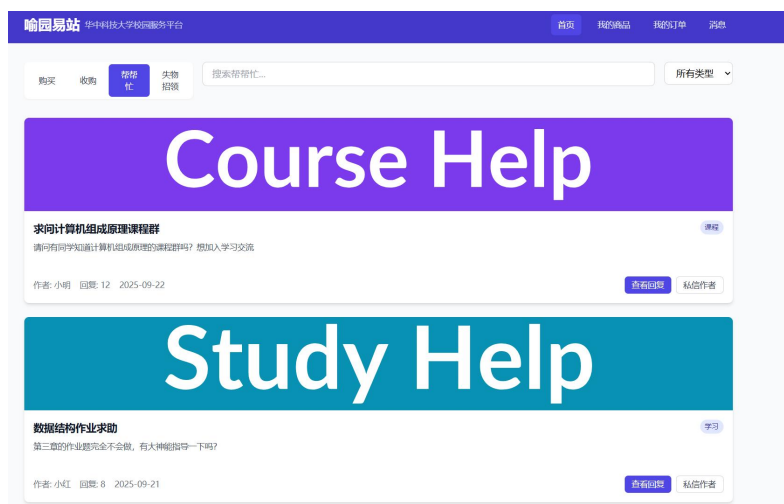


图 2-6 原型网站帮帮忙模块

在“失物招领”板块，用户可选择“寻物启事”或“招领信息”两类，填写物品名称、丢失 / 招领地点、描述、图片、联系方式。系统在列表中以卡片形式展示线索帖，支持用户提交线索和私信联系。整体界面设计简洁、响应式，采用 Tailwind CSS 快速组织布局，适应多种屏幕尺寸。根据项目说明，“多模式切换”功能即位于首页导航之上，实现了功能间无缝切换。

通过该原型设计，需求分析所提出的功能点——包括交易流程、筛选功能、私信聊天、状态管理、模式切换等——得以更直观地呈现，用户体验路径更为清晰，也为后续开发阶段提供了可执行的 UI / UX 指引。原型设计既具备展示功能，也具备可测试的交互流，为团队早期验证需求合理性提供了重要支撑。

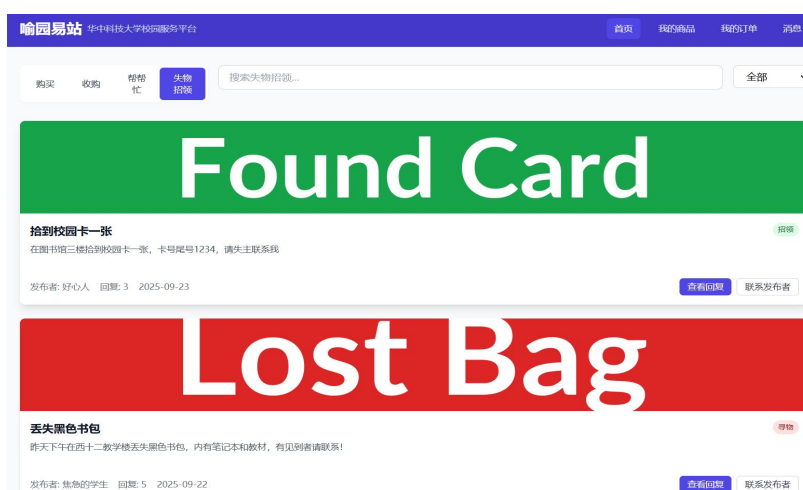


图 2-7 原型网站失物招领模块

## 3 概要设计和详细设计

### 3.1 系统结构

在明确了系统需求之后，我们为“喻园易站”平台设计了完整的系统结构，其目标是将交易、互助问答、失物招领三大核心功能模块有机整合，同时保证平台具有良好的扩展性、维护性与性能。整体来看，系统结构可分为以下几个主要模块：用户认证模块、交易服务模块、问答社区模块、失物招领模块、消息通信模块、后台管理模块、数据存储模块和接口服务模块。

用户认证模块负责完成用户注册、登录、角色判断（普通用户或管理员）、用户资料管理等功能。交易服务模块负责处理交易帖子的发布、浏览、筛选、下单、状态变更、订单管理以及多图上传功能。问答社区模块涵盖用户发布求助帖、帖子讨论、回答、评论、采纳、私信功能。失物招领模块则专注于“寻物启事”与“招领信息”的发布、线索提交、讨论回复、私聊发布者等交互流程。消息通信模块提供实时聊天、通知推送，基于 WebSocket 或轮询机制。后台管理模块为管理员提供用户与内容审核、举报处理、统计分析等功能。数据存储模块使用 MySQL 等关系型数据库或云存储服务来可靠地保存用户数据、帖子数据、订单数据、聊天记录、图片资源等。接口服务模块即后端 API 层，负责所有模块对外服务的统一接口，实现不同模块之间以及前端与后端之间的协同。

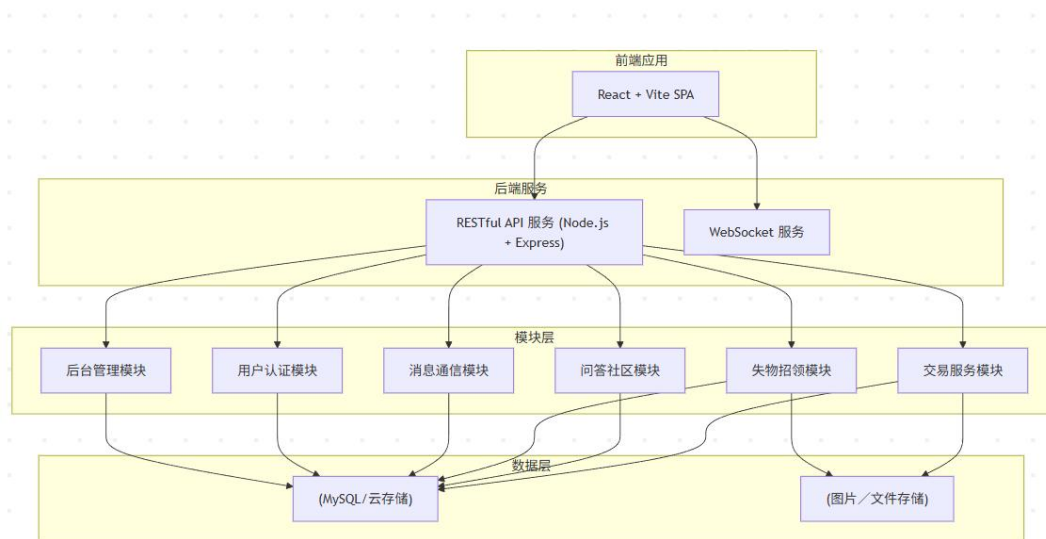


图 3-1 系统结构图

这张结构图展示了系统从前端用户界面到后端服务，再到模块及数据层的流向。前端通过 REST 接口访问后端服务，同时也通过 WebSocket 实现实时通信。各模块清晰分工，但共享数据存储与文件存储设施，从而保证统一的存储管理。后台管理模块直接调用数据层以进行内容审核和统计。

接口设计方面，每一个功能模块都提供相应的 API 路径。例如用户认证模块提供 POST /api/auth/register、POST /api/auth/login、GET /api/auth/profile 等接口；交易服务模块提供 POST /api/posts（发布）、GET /api/posts（列表查询）、POST /api/orders（下单）、PATCH /api/orders/:id（状态更新）等；问答社区模块提供 POST /api/questions、GET /api/questions/:id/replies、POST /api/questions/:id/replies、POST /api/messages 等；失物招领模块类似。所有接口均返回统一的 JSON 格式，并通过 JWT 认证、统一错误处理及分页机制实现高效、可维护的服务接口。

### 3.2 设计视图

本系统采用“前后端分离 + REST API + WebSocket”的典型三层结构：前端（Vite + React + Axios + Tailwind）通过 HTTP 与后端（Express + MySQL + Multer + JWT + Nodemailer + ws）交互，消息模块通过同端口的 /ws 建立 WebSocket 实时通道；图片等用户上传文件保存在后端的 uploads 目录并以静态资源方式暴露。

在后端内部，各业务通过“服务模块化”的方式组织：认证、帖子/跑腿、订单、消息、回复分别作为路由分组；共享的数据库连接池、文件存储与工具函数提供横向支撑。所有对数据一致性有要求的流程（如下单、接单、确认完成）均以事务包裹，并在关键读取上使用 SELECT ... FOR UPDATE 防止并发写冲突。敏感信息（跑腿的起止地点与私密备注）做了“按身份可见”的控制：仅发单人与接单人可见，列表/详情接口都会根据 JWT 判断是否脱敏。

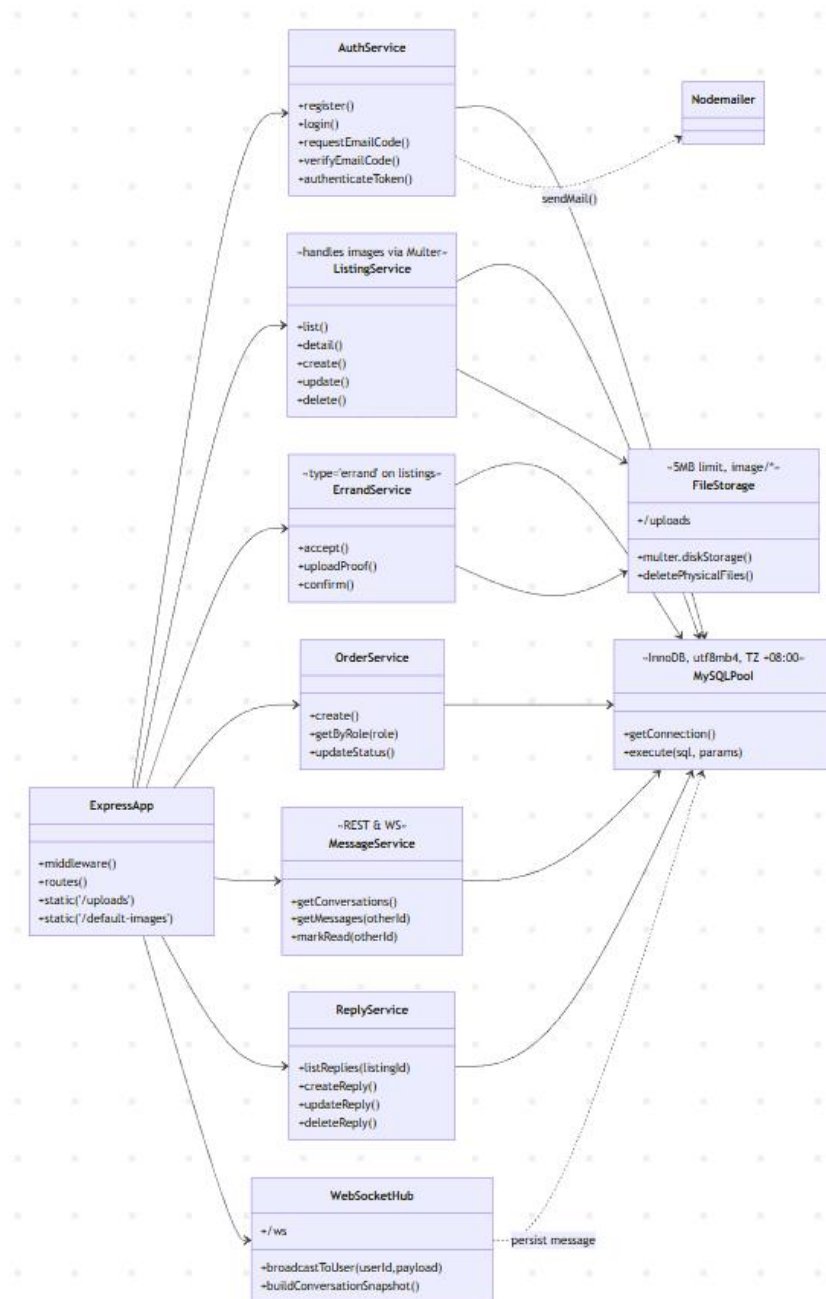


图 3-2 设计视图

### 3.3 关键数据结构定义

本系统围绕“用户-帖子/任务-互动（订单 / 消息 / 回复）”构建数据层。核心数据对象既要覆盖普通的二手商品与收购、失物招领、讲座代课，又要支持新扩展的“跑腿代办”任务（含接单流程与完成凭证），并提供社交（关注 / 收藏）与通讯（站内私信、二级评论）能力。下面依次说明：系统处理的数据类别；每个类别的字段与类型；它们之间的关联关系（用 ER 图 + 结构图直观展示）；

以及这些结构在业务流程中的语义。

### 3.3.1 数据类型总览与字段说明

以下图 3-3 列出了后端数据库的主要表结构与字段(类型为 MySQL 语义), 并说明每一字段的用途与约束。复合结构(如跑腿订单嵌入在 listings 表)以“分组”方式标注。

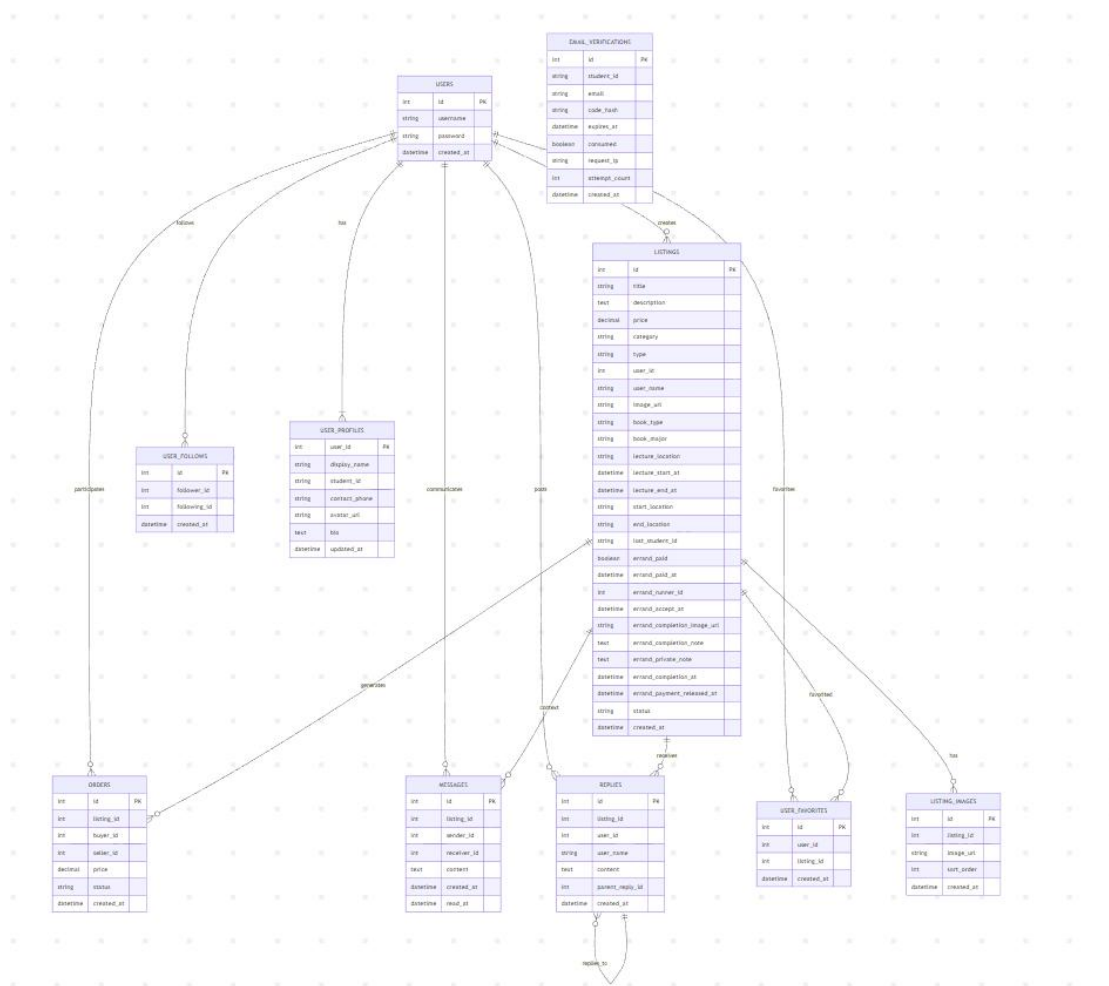


图 3-3 主要数据结构和关系

#### 1. USERS

id (INT, PK): 用户主键。

username (VARCHAR, UNIQUE): 登录名; 若使用学号注册, 可能为学号小写。

password (VARCHAR, HASH): bcrypt 哈希后的密码。

created\_at (TIMESTAMP): 注册时间。

用途：身份与鉴权的最小实体。多数业务冗余保存 username 以减少 JOIN 次数。

## 2. USER\_PROFILES

user\_id (INT, PK, FK): 与 USERS 一对一。

display\_name: 昵称。

student\_id: 学号（可绑定，绑定后不可改）。

contact\_phone: 联系方式（可选）。

avatar\_url: 头像地址（自定义或默认图）。

bio: 个人简介。

updated\_at: 最后更新时间。

用途：把变更频繁、可选的扩展字段从 USERS 分离，减小基础表的写放大。

## 3. USER\_FOLLOWS

follower\_id: 关注发起者。

following\_id: 被关注者。

created\_at: 关注时间。

用途：社交关系。唯一约束 (follower\_id, following\_id) 防重复关注。支持统计：关注数、粉丝数、互关状态。

## 4. USER\_FAVORITES

user\_id: 收藏者。

listing\_id: 被收藏帖子。

created\_at: 收藏或更新收藏时间（重复收藏会刷新时间做排序）。

用途：个人收藏列表，允许用户快速回访感兴趣的帖子。

## 5. EMAIL\_VERIFICATIONS

student\_id: 学号。

email: 映射出的校邮箱（如 u20xxxxxxx@hust.edu.cn）。

code\_hash: 验证码哈希（避免明文存储）。

expires\_at: 过期时间（10 分钟）。

consumed: 是否已使用。

request\_ip: 请求来源 IP。

attempt\_count: 错误尝试次数（暴力防护）。

created\_at: 申请时间。

用途：支持学号邮箱验证注册；安全性重点在哈希与次数限制。

## 6. LISTINGS（核心：多类型统一）

公共字段：

id, title, description, price, category（在模块内部的细分：books / lecture / service / lost\_found 子类）、type（sale / acquire / help / lostfound / errand）。

user\_id / user\_name: 发布者与冗余用户名快照。

image\_url: 封面图（第一张）。

status: available / in\_progress / completed 等。

created\_at: 发布时间。

扩展字段（按模块）：

图书：book\_type（课内教材等），book\_major（专业）。

代课讲座：lecture\_location, lecture\_start\_at, lecture\_end\_at。

跑腿：start\_location / end\_location（地点可能历史为 JSON 数组合并，现已标准化），errand\_paid / errand\_paid\_at（模拟支付标记），errand\_runner\_id / errand\_accept\_at（接单时间及时间），errand\_completion\_image\_url + errand\_completion\_note（完成凭证），errand\_private\_note（发单者与接单者间的隐私备注，不对外展示），errand\_completion\_at（接单者提交完成时间），errand\_payment\_released\_at（发单人确认时间）。

失物招领：lost\_student\_id（潜在失主学号，用于通知）。

用途：采用“单表多字段”而非拆分多张子表，以降低跨类型筛选/统计的复杂度；通过字段稀疏性 + 索引控制性能。

## 7. LISTING\_IMAGES

listing\_id: 父帖子。

image\_url: 图片 URL。

sort\_order: 排序号 (0 为封面)。

created\_at: 上传时间。

用途: 多图支持。更新时通过 keepImageIds + 新文件插入重算排序。

## 8. ORDERS

listing\_id: 关联商品帖子 (仅适用于普通出售/收购, 不含跑腿)。

buyer\_id / seller\_id: 交易双方。

price: 下单时价格快照 (避免后续修改影响)。

status: to\_pay → to\_ship → to\_receive → completed / cancelled。

created\_at: 下单时间。

用途: 交易流程的最小状态机; 与 listings.status 联动 (下单后变 in\_progress, 完成/取消后同步回写)。

跑腿订单不入 ORDERS 表, 直接在 LISTINGS 中以 errand\_\* 字段驱动状态与流程, 避免双轨数据冗余。

## 9. REPLIES

listing\_id: 归属帖子。

user\_id / user\_name: 作者与用户名快照。

content: 文本内容。

parent\_reply\_id: 二级评论父节点 (只允许一层嵌套)。

created\_at: 发布时间。

用途: 树宽不深, 采用 “单表 + parent\_reply\_id” 即可, 无需闭包表或路径枚举。

## 10. MESSAGES

sender\_id / receiver\_id: 消息双方。

listing\_id: 可选关联上下文 (从某个帖子发起的私信)。

content: 消息正文。

created\_at / read\_at: 发送时间与已读时间。

用途：即时通讯。会话摘要通过聚合 MAX(created\_at) 与未读计数 SUM 条件生成；WebSocket 推送实时刷新。

### 3.3.2 数据对象之间的语义关联与使用场景

系统典型操作链条体现为：用户（USERS + USER\_PROFILES）发布不同类型的 LISTINGS；其他用户通过浏览与筛选触发收藏（USER\_FAVORITES）、关注（USER\_FOLLOWS）、私信（MESSAGES）、评论（REPLIES），并在“出售/收购”场景下生成 ORDERS；在“跑腿代办”场景直接使用 LISTINGS 内嵌的 errand\_\* 字段完成接单与确认流程。

为了更直观展示跑腿订单如何“嵌入”普通帖子结构，下面用一个结构视图突出 LISTINGS 的分层字段：



图 3-4 Listings 分层结构

下面对重点场景的文本关联进行一定说明：

普通交易: LISTINGS 里 `type='sale'/'acquire'`; 用户点击“立即购买” → 生成 ORDERS 记录 → 状态向前推进; 完成后 `ORDERS.status=completed` 且 `LISTINGS.status` 同步为 `completed`。

跑腿代办: 不生成 ORDERS。接单人写入 `errand_runner_id`, 流程与凭证通过 `errand_*` 字段进阶; 完成后由发单人确认并写入 `errand_payment_released_at`。

失物招领: `lost_student_id` 用于异步通知潜在失主 (发送 MESSAGES 系统提示)。

图书/讲座扩展: 通过 `book_type/book_major` 与 `lecture_*` 字段支持复杂筛选 (包括时间区间重叠判定)。

评论与私信: REPLIES 与 MESSAGES 都以 `listing_id` 作为上下文 (可选), 区别在于公开 (树形) vs 私密 (双向流 + 已读)。

### 3.3.3 数据关联的访问模式与性能注意

本项目在设计中采用了以下策略保证可维护与扩展性:

冗余快照字段 (`user_name`、`sender/receiver` 用户名) 降低高频列表场景中的 JOIN 开销。

跑腿与普通交易合并在 LISTINGS, 一次查询即可混排展示各种类型, 提高首页性能。

通过 `listing_images` 独立出多图, 保证 LISTINGS 主表行宽可控; 更新图片时用事务 + `keepImageIds` 精准删除物理文件。

消息会话摘要的 SQL 使用 `MAX(CONCAT(created_at, '\x1f', content))` 提取最新内容, 避免子查询或窗口函数, 向下兼容。

`errand_*` 字段组合形成“内嵌状态机”: 不需额外订单表即可表达接单、上传凭证、发单人确认三阶段。

### 1. 多模块内容展示与主题适配算法（前端）

算法思想：基于 `moduleThemes.js` 定义的主题配置（颜色、文案、布局规则），通过类型映射实现动态样式注入，采用 “数据驱动视图” 模式，根据内容类型自动选择对应组件（`ListingCard/InfoCard`）

## 2. 跑腿任务状态流转算法（前后端协同）

算法思想：基于状态机模型设计状态流转规则，通过后端权限校验确保操作合法性，前端根据状态动态展示可用操作（如已接单后隐藏“接单”按钮）

待接单 → [跑腿者接单] → 已接单 → [上传完成证明] → 待确认 → [发布者确认] → 已完成  
 \ [超时/取消] → 已关闭

**核心场景：**通过 WebSocket 实时同步会话未读消息数，优化用户体验。

性能，基于消息时间戳判断未读状态，避免重复计数

连接初始化：用户进入消息页，前端建立 WebSocket 连接，携带 JWT 令牌验证，后端关联用户 ID 与连接实例，存储于 userSockets 映射表

已读同步机制：用户点击会话，前端调用 POST

/api/messages/conversations/:userId/read，携带当前时间戳，后端更新数据库中该会话的 last\_read\_time，前端重置对应会话 unreadCount 为 0

断线重连处理：网络恢复后自动重连，重连成功后拉取 lastReadTime 之后的消息补全计数

#### 4. 图片上传与资源管理算法（Multer + 路径解析）

核心场景：处理帖子 / 任务的图片上传、存储与访问，支持多图上传与编辑时的图片保留。

算法思想：采用 Multer 中间件处理文件上传，结合哈希命名避免冲突，前端通过统一路径解析函数处理资源访问，兼容不同部署环境

流程步骤：

选择图片 → FormData 封装 → 后端 Multer 处理 → 哈希命名存储 → 返回相对路径 → 前端 resolveAssetUrl 解析 → 渲染图片

#### 5. 订单状态管理算法（多角色视图）

核心场景：根据用户角色（买家 / 卖家 / 跑腿）展示不同的订单状态与可执行操作。

算法思想：基于角色权限过滤订单数据，通过状态枚举控制操作按钮显示逻辑，采用“单一数据源”模式，后端统一返回全量状态，前端按角色筛选展示状态展示视图：

买家视图：显示 “待支付” → “已支付” → “待收货” → “已完成”

卖家视图：显示 “待接单” → “已接单” → “已发货” → “已完成”

跑腿视图：显示 “待接单” → “已接单” → “待确认” → “已完成”

状态更新视图：

点击操作按钮（如 “确认支付”），前端调用 PUT /api/orders/:id/status

买家 待支付 → 已支付 → 已完成 支付、取消、确认收货

卖家 待接单 → 已接单 → 已发货 → 已完成 接单、标记发货

跑腿 待接单 → 已接单 → 待确认 → 已完成 接单、上传证明、查看进度

### 3.5 数据管理说明

本系统的数据分为“结构化业务数据”和“非结构化文件数据”两大类：结

结构化数据使用 MySQL 进行持久化存储（以 listings 为中心，关联 users、orders、messages、replies 等表）；非结构化数据（图片等）使用本机文件系统存储，并通过静态资源映射对外提供访问 URL。数据访问由后端 Node.js（Express + mysql2 连接池）统一调度，在关键业务（发帖编辑、下单、跑腿接单与确认、图片变更、消息发送等）中使用事务与行级锁保证一致性。

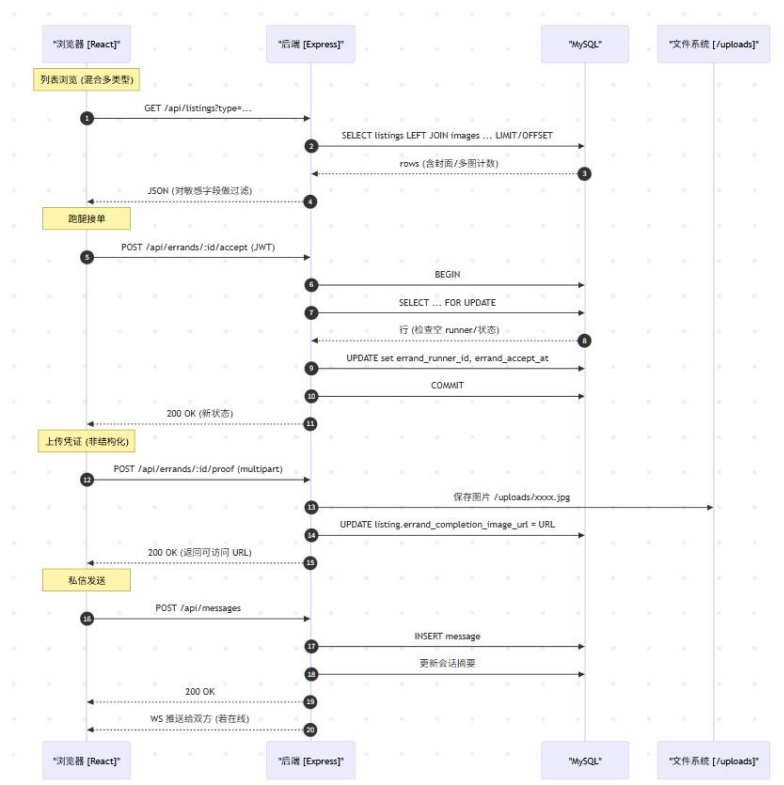


图 3-5 数据管理时序图

在读密集场景（首页列表、我的订单/发布、消息会话摘要）中优先减少 JOIN 次数与字段体积（如冗余 user\_name 快照、封面直存、图集分表），在写敏感场景（并发接单、订单状态推进、图片替换）中强调“短事务 + 行级锁”。

### 3.5.1 存储介质与数据落盘

系统仅依赖两类基础设施：MySQL 与磁盘文件系统。各存储项职责如下。

表 3-1 系统数据存储结构一览表

| 存储介质       | 位置 / 名称                      | 存放内容      | 说明                                                                       |
|------------|------------------------------|-----------|--------------------------------------------------------------------------|
| MySQL      | users, user_profiles         | 身份与扩展资料   | 密码以 bcrypt 哈希保存; 用户资料与学号唯一绑定                                             |
| MySQL      | listings                     | 核心帖子 / 任务 | 单表支持多类型 (图书 / 讲座 / 跑腿 / 失物等); 跑腿相关状态字段以 errand_ 为前缀 (如 errand_runner_id) |
| MySQL      | listing_images               | 多图支持      | 封面图 URL 存于 listings.image_url; 图集及排序信息存储在此表                              |
| MySQL      | orders                       | 普通买卖订单    | 仅用于商品交易类订单; 跑腿任务不进入此表, 避免双轨冗余                                            |
| MySQL      | messages                     | 站内私信      | 用于实现会话摘要、未读消息计数和消息历史                                                     |
| MySQL      | replies                      | 评论 / 二级回复 | 支持一级嵌套回复, 通过 parent_reply_id 字段关联上级评论                                    |
| MySQL      | user_follows, user_favorites | 关注与收藏关系   | 使用组合唯一索引 (如 (user_id, target_id)) 防止重复操作                                 |
| MySQL      | email_verifications          | 邮箱验证码记录   | 存储验证码的哈希值、过期时间、验证尝试次数, 提升安全性                                             |
| FileSystem | /uploads                     | 用户上传的图片   | 通过 multer 中间件处理上传, 静态服务映射为 https://.../uploads/xxx                       |
| FileSystem | /default-images              | 默认占位图片    | 静态资源目录, 无后端上传逻辑, 仅用于前端 fallback 显示                                       |

文件访问通过 Express 静态中间件将磁盘路径映射为 HTTP 路径, 对外表现为稳定 URL; 前端统一用 resolveAssetUrl 将相对路径拼成完整可访问地址。图片文件名使用时间戳与随机数组合, 避免冲突; 当更新图片或替换凭证时, 业务层会在事务完成后删除被置换的旧文件, 减少磁盘堆积。



图 3-6 图片上传流程图

字符集统一为 utf8mb4（兼容 emoji），时间均以服务器本地时间落库（字段 DATETIME/TIMESTAMP）。索引针对高频检索字段建立（如 listings(user\_id, type, status), listing\_images(listing\_id, sort\_order), messages(sender\_id, receiver\_id, created\_at), listings(errand\_runner\_id) 等），以降低分页与模块筛选的延迟。

### 3.5.2 数据访问路径与一致性策略

访问统一通过 Express 路由进入业务层，使用 mysql2 的连接池执行 SQL。读写策略遵循“读快、写稳”的取舍：普通读取尽量避免复杂 JOIN；关键写入使用短事务与必要的行锁。

#### 1. 典型读取路径

列表查询仅选择必要字段（标题、封面、价格、状态、时间等）并按模块增量拼接筛选（图书类型/讲座时间/跑腿地点）。隐私字段（如跑腿 errand\_private\_note、完成凭证等）仅在“参与双方或发帖者”上下文下才会返回，其他情况下在 API 层过滤掉。分页采用 LIMIT/OFFSET，前端维护页码/滚动增量加载。

#### 2. 典型写入路径

发帖/编辑使用事务：插入或更新 listings 后批量维护 listing\_images 表，并重算封面；使用 keepImageIds 保留未删除的图集并在事务提交后清理已删除文件。跑腿接单/上传凭证/确认完成三个环节均为短事务：

接单：SELECT ... FOR UPDATE 读取该行，校验 runner 为空与状态可接，再更新 errand\_runner\_id 与时间戳，避免并发踩踏。

上传凭证：保存图片到 /uploads，写入 errand\_completion\_image\_url/

errand\_completion\_note/errand\_completion\_at；如有旧凭证，成功提交后删除旧文件。

确认完成：由发单人触发，写入 errand\_payment\_released\_at 并将状态推进为 completed（资金释放为模拟字段）。

普通订单（sale/acquire）在创建与状态推进时同步更新 listings.status，保证“订单视图”和“帖子视图”的一致。

#### 3. 事务与锁

仅在需要序列化的关键更新使用事务与行锁（FOR UPDATE），遵循“锁粒

度小、持有时间短”的原则，避免广域锁导致的吞吐下降。文件系统操作不放入数据库事务中，采用“先写库再清理旧文件”的补偿式策略，确保数据库一致性优先。

#### 4. 会话消息与已读

消息写入时直接 INSERT，在查询“会话摘要”时按双方（无向或有向）聚合，取 MAX(created\_at) 作为最新时间、SUM(read\_at IS NULL AND receiver=me) 作为未读数；若对端在线，通过 WebSocket 即时下发，客户端再合并本地会话状态。

下图以跑腿代办任务为例，展示了状态流转和数据更新处理过程。

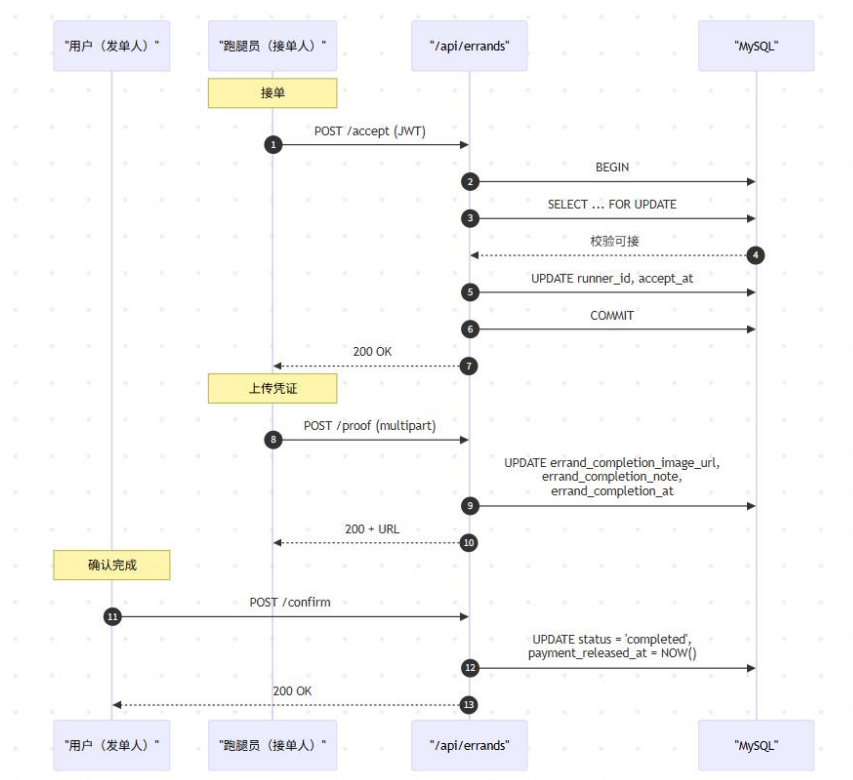


图 3-7 跑腿任务状态流转时序图

### 3.5.3 运行与运维

数据演进通过后端“幂等初始化/迁移”函数执行：在服务启动时检测并补齐缺失字段与索引（如 errand\_\*、start/end\_location 等），采用 ALTER TABLE ... ADD COLUMN IF NOT EXISTS / 先查后加 的方式保证可重复执行，避免手工

脚本造成漂移。升级前建议先做全库备份（mysqldump），升级后对关键表结构做二次校验。

备份与持久化方面，数据库使用 RDS 或自建实例按日备份；文件系统的 /uploads 需在部署时挂载持久卷（或定期同步到对象存储），避免容器重建导致图片丢失。前端引用的图片均通过 URL 访问，不依赖进程内存。

安全与合规方面。密码存 bcrypt 哈希，JWT 保存在前端的 localStorage，仅在 HTTPS 环境下部署对外服务。邮箱验证码以哈希 + 过期时间存储，附带尝试次数与来源 IP，防止暴力破解。API 会在返回层屏蔽不应外露的隐私（如跑腿私密备注）并校验资源所有权（仅资源拥有者可编辑、被接单后不允许编辑）。上传仅接受图片 MIME，并限制文件大小。

性能与扩展方面，列表页以“稀疏列 + 冗余快照”减少 JOIN；热点筛选字段建索引；大分页通过前端滚动加载与最后一条游标优化（可按需引入）。消息会话摘要用轻量聚合保持查询简单。若需进一步扩展，优先引入读写分离/连接池扩容，再考虑引入缓存（如 Redis）加速会话与计数类查询。图片可迁移到对象存储 + CDN，前端 URL 解析逻辑不变。

## 4 实现与测试

### 4.1 实现环境与代码管理

#### 4.1.1 实现环境

##### 1. 本地开发环境

- (1) 操作系统: Windows 11 操作系统
- (2) 开发工具: Visual Studio Code, 用于前后端代码编写、调试及版本控制集成。
- (3) 版本控制工具: Git, 用于代码版本管理, 支持本地开发与 Gitee 的同步。
- (4) 终端工具: VS Code 内置终端, 执行命令行操作

##### 2. 开发语言与核心技术栈:

- (1) 后端: Node.js), 采用 Express 框架构建 RESTful API 与 WebSocket 服务, 使用 mysql2/promise 操作 MySQL 数据库, 通过 JWT 实现身份认证, bcrypt 处理密码加密, multer 实现文件上传, nodemailer 发送邮箱验证码。
- (2) 前端: React 18/19, 基于 Vite 构建工具开发, 使用 Axios 进行 API 通信, Tailwind CSS 实现样式设计, 通过 Context API 管理全局状态(如登录状态、消息提示等)。

数据库: MySQL 8.x, 用于存储用户信息、帖子数据、消息记录等业务数据。

##### 3. 部署环境配置

- (1) 服务器环境: 阿里云云服务器, 用于部署前后端应用及数据库服务。
- (2) 操作系统: Ubuntu 22.04
- (3) 网络配置: 安全组需放行必要端口(如后端服务端口 3000、MySQL 端口 3306、SMTP 端口 465/587 等), 确保服务可访问。
- (4) 部署工具与服务:

后端部署: 通过 PM2 作为 Node.js 进程守护工具, 确保后端服务持续运行。

前端部署: 前端构建产物(dist 目录)通过 Nginx 作为静态资源服务器托管。

反向代理: 使用 Nginx 实现反向代理, 将/api 请求转发至后端服务端口, /uploads 映射至后端上传目录, /ws 支持 WebSocket 协议升级与连接保持。

数据库部署：MySQL 8.x 服务部署于阿里云服务器，配置远程访问权限（或仅允许本地后端服务访问以提升安全性）。

#### 4.1.2 代码签入记录

使用码云平台进行代码管理，下面分别展示了仓库数据统计图和仓库网络图。

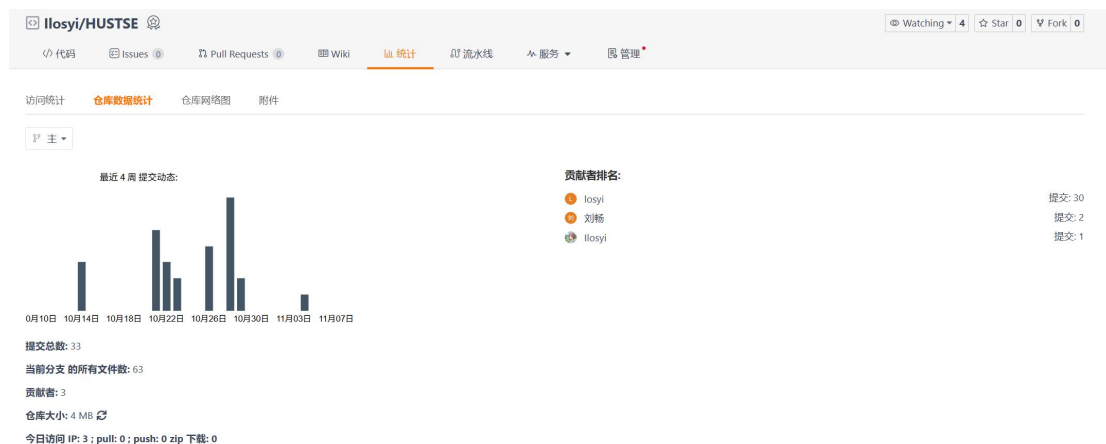


图 4-1 码云平台仓库数据统计

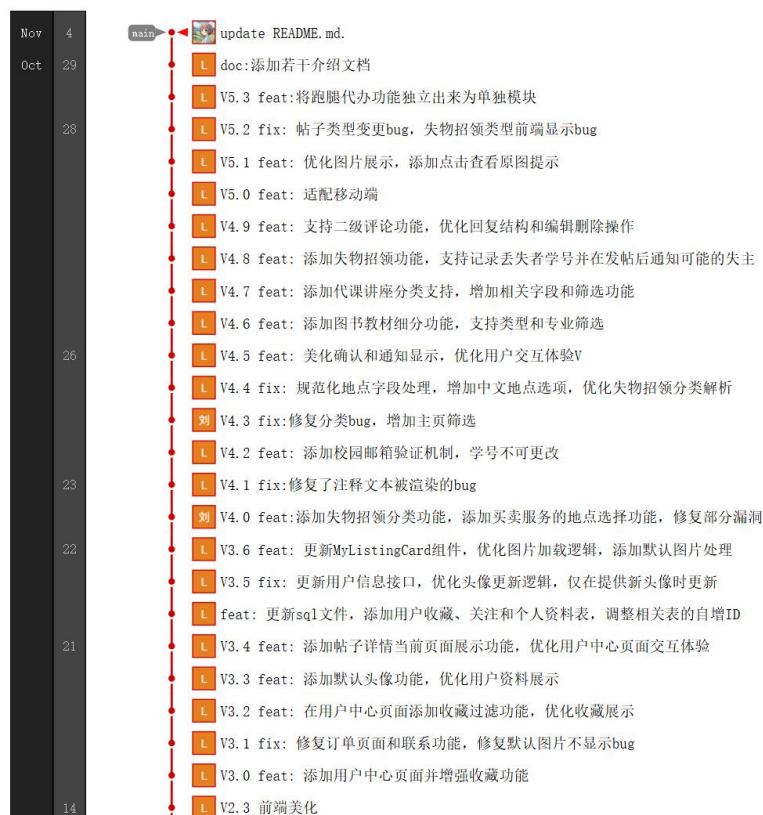


图 4-2 码云平台仓库网络图（版本迭代记录）

## 4.2 关键函数说明

### 4.2.1 核心工具函数（后端）

#### 1. 文件处理相关函数

**resolveUploadAbsolutePath(value):** 将 `/uploads/` 开头的相对路径转换为服务器磁盘绝对路径，同时验证路径安全性（防止目录越权访问）。

**gatherUploadedImages(req):** 合并请求中通过 `images[]` 或 `image` 字段上传的图片文件，统一格式为数组，便于后续处理。

**buildImageUrl(file):** 根据上传文件信息生成对外可访问的 URL（如 `/uploads/202405/img123.jpg`），供前端渲染使用。

**deletePhysicalFiles(urls):** 将图片 URL 转换为绝对路径后删除对应物理文件，用于编辑帖子时清理废弃图片，避免存储冗余。

模块关联：支撑图片上传（发布 / 编辑帖子、上传跑腿证明）功能，在 `POST /api/listings`、`PUT /api/listings/:id` 等接口中被调用。

#### 2. 用户认证相关函数

**authenticateToken(req, res, next):** JWT 鉴权中间件，从请求头提取 `Authorization` 令牌并验证，通过后将用户信息注入 `req.user`，保护需要登录的接口。

**tryDecodeToken(token):** 非强制解析令牌，用于 `WebSocket` 连接时验证用户身份（`/ws` 路径），避免未登录用户建立连接。

**buildEduEmailFromStudentId(studentId):** 根据学号生成校园邮箱（如 `studentId@hust.edu.cn`），支撑邮箱验证码注册流程。

模块关联：所有受保护接口（如 `/api/orders`、`/api/users/me`）均通过 `authenticateToken` 校验；邮箱注册流程依赖邮箱生成函数。

#### 3. WebSocket 辅助函数

**registerSocket(userId, ws):** 将用户 ID 与 `WebSocket` 实例关联存储，用于定

向消息推送。

**broadcastToUser(userId, data):** 向指定用户的所有在线连接推送消息（如新消息通知、会话更新）。

**buildConversationSnapshot(userId):** 生成用户的会话摘要（含最后一条消息、未读数），用于消息页初始化加载。

模块关联：支撑 `MyMessagesPage` 的实时通信功能，在消息发送、已读标记时触发会话更新广播。

## 4.2.2 业务逻辑函数（后端）

### 1. 跑腿任务流程函数

**acceptErrand(req, res):** 处理跑腿任务接单逻辑，验证任务状态（必须为“待接单”）并更新 `errand_runner_id` 和接单时间，在 `POST /api/errands/:id/accept` 接口中调用。

**submitErrandProof(req, res):** 接收跑腿者上传的完成证明（图片 + 备注），更新任务状态为“待确认”，关联 `POST /api/errands/:id/proof` 接口。

**confirmErrandCompletion(req, res):** 验证发布者身份后，确认任务完成并更新支付状态，对应 `POST /api/errands/:id/confirm` 接口。

模块关联：串联跑腿任务从接单到完成的全流程，在 `MyListingsPage` 中被前端操作触发。

### 2. 订单状态管理函数

**getOrdersByRole(req, res):** 根据角色（买家 / 卖家 / 跑腿）筛选订单，返回对应视图的数据（如买家只看自己购买的订单），支撑 `GET /api/orders?role=xxx` 接口。

**updateOrderStatus(req, res):** 验证操作权限后更新订单状态（如“待支付” → “已支付”），并记录状态变更时间，对应 `PUT /api/orders/:id/status` 接口。

模块关联：为 `MyOrdersPage` 提供数据与状态更新能力，不同角色视图通过角色参数区分。

### 4.2.3 前端核心函数

#### 1. API 封装函数 (src/api/index.js)

`resolveAssetUrl(relativePath)`: 将后端返回的相对路径（如 `/uploads/xxx.jpg`）转换为可访问的绝对路径（拼接 API 基址），确保图片在不同环境（开发 / 生产）下正常加载。

拦截器函数：请求拦截器自动添加 `Authorization` 头；响应拦截器处理令牌过期等错误，统一弹出提示。

模块关联：所有前端页面通过该模块发起请求，如 `ListingCard` 用 `resolveAssetUrl` 渲染图片。

#### 3. 状态管理函数 (context/AuthContext.jsx)

`login(credentials)`: 发送登录请求，成功后将令牌和用户信息存入 `localStorage` 并更新上下文状态。

`requestEmailCode(studentId) / verifyEmailRegister(data)`: 对接邮箱验证码流程，支撑 `RegisterPage` 的注册功能。

模块关联：全局提供登录状态，`LoginPage`、`RegisterPage` 调用登录 / 注册函数，其他页面通过上下文获取用户信息。

### 4.2.4 模块功能实现逻辑

图片上传模块：前端通过 `PostModal` 选择图片，调用 API 层上传；后端经 `gatherUploadedImages` 处理后，用 `buildImageUrl` 返回路径，前端通过 `resolveAssetUrl` 渲染。

跑腿任务模块：用户发布任务后，跑腿者通过 `MyListingsPage` 调用 `acceptErrand` 接单；完成后调用 `submitErrandProof` 上传证明；发布者通过 `confirmErrandCompletion` 确认，全流程由状态函数控制合法性。

消息模块：`MyMessagesPage` 建立 `WebSocket` 连接（`registerSocket`），收发消息时触发 `broadcastToUser` 同步会话状态，未读计数依赖 `buildConversationSnapshot` 初始化数据。

## 4.3 测试计划和测试用例

软件测试是保证软件质量、确认软件功能符合需求规格说明书的重要环节。本节首先简述常用的测试方法，随后选取本项目中逻辑最为复杂、交互性最强的两个核心模块——“跑腿代办服务模块”与“即时通讯（WebSocket）模块”，详细描述其测试过程、数据选取及测试结果。

### 4.3.1 常用软件测试方法概述

在本项目中，我们采用了多种测试方法相结合的策略，以确保系统的健壮性：

#### 1. 黑盒测试 (Black-box Testing):

定义：把程序看作一个不能打开的黑盒子，不考虑程序内部逻辑结构，只根据需求规格说明书，检查程序功能是否符合要求。

应用：主要用于前端 UI 测试和 API 接口的功能性验证（如输入正确的用户名密码能否登录）。

#### 2. 白盒测试 (White-box Testing):

定义：基于程序内部的逻辑结构设计测试用例，对程序的逻辑路径进行覆盖测试。

应用：主要用于后端复杂逻辑的单元测试，例如跑腿接单时的并发锁逻辑（SELECT ... FOR UPDATE）和状态机流转。

#### 3. 集成测试 (Integration Testing):

定义：将模块组装起来进行测试，检查模块间的接口是否正确。

应用：测试前端 React 组件与后端 Express API 的交互，以及后端与 MySQL 数据库、文件系统（Multer）的集成。

### 4.3.2 模块一：跑腿代办服务模块测试

#### 1. 模块功能与设计目标

功能描述：该模块支持用户发布跑腿需求（买东西、取快递等），其他用户（Runner）接单、上传完成凭证，最后由发单人确认完成并释放酬劳。

设计目标：

状态一致性：严格遵循 available (待接单) -> in\_progress (进行中) ->

completed (已完成) 的状态流转。

并发安全：防止多个 Runner 同时接同一个单（超卖问题）。

隐私保护：errand\_private\_note（私密备注，如取件码）仅对发单人和接单人可见，其他人不可见。

2. 测试数据选取与分析为了验证上述目标，设计覆盖正常流程和异常边界的数据：

用户数据：用户 A（发单人，ID: 101）、用户 B（接单人，ID: 102）、用户 C（路人 / 恶意抢单者，ID: 103）。

帖子数据：帖子 ID: 67（类型 type='errand', 状态 available, 含私密备注 "取件码 8888"）。

边界数据：尝试接状态为 in\_progress 的订单；非接单人尝试查看私密备注。

3. 测试计划与用例测试环境：Chrome 浏览器（前端），Postman（API 接口并发测试），MySQL 数据库。

表 4-1 测试计划表

| 用例编号      | 测试项     | 前置条件                         | 测试步骤                                                 | 预期结果                                                                  | 实际结果     | 结论 |
|-----------|---------|------------------------------|------------------------------------------------------|-----------------------------------------------------------------------|----------|----|
| TC-ERR-01 | 正常接单流程  | 用户 A 发布跑腿单 67, 状态为 available | 1. 用户 B 登录<br>2. 点击“接单”按钮（调用 /api/errands/67/accept） | 1. 接口返回 200 OK<br>2. 数据库中 errand_runner_id 更新为 1023. 状态变为 in progress | 与预期一致    | 通过 |
| TC-ERR-02 | 并发抢单测试  | 跑腿单 5002 刚发布                 | 1. 使用 Postman 模拟用户 B 和 C 几乎同时发送接单请求                  | 1. 仅一个用户返回成功<br>2. 另一用户返回 400/409 错误<br>3. 数据库 runner_id 唯一           | 成功拦截并发请求 | 通过 |
| TC-ERR-03 | 隐私字段可见性 | 订单 67 已被 B 接单                | 1. 用户 C 调用详情接口<br>2. 用户 B 调用同一接口                     | 1. 用户 C 响应中私密备注隐藏<br>2. 用户 B 可见“取件码 8888”                             | 身份过滤正确   | 通过 |
| TC-ERR-04 | 上传凭证与确认 | 订单 67 进行中                    | 1. 用户 B 上传图片凭证<br>2. 用户 A 确认完成                       | 1. 凭证 URL 写入数据库<br>2. 状态变为 completed<br>3. payment_released_at 有时间戳   | 流程顺畅     | 通过 |

#### 4. 测试过程

### (1)前置准备：获取身份令牌 (Token)

所有后续操作都需要登录态，因此第一步必须获取 Token。

请求方法: POST

URL: `http://localhost:3000/api/auth/login`

Body (选择 raw -> JSON):

```
{
  "username": "losyi",
  "password": "111111"
}
```

响应结果

```
{
  "accessToken":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NywidXNlcm5hbWUiOiJsb3N5aSI6Im1hdCI6MTc2NDU4MDg4NywiZmxhZSI6bnZyY3Mjg3fQ.mRt3l9Q4U5KAHuHpyF78cIgc1YkFnIEil9Z7v1aQDHI",
  "user": {
    "id": 7,
    "username": "losyi"
  }
}
```

操作: 点击 Send, 复制响应中的 token 字符串（不要带引号）。

技巧: 在 Postman 的 "Environments" 中设置一个变量 `jwt_token`, 将值填入, 后续请求用 `{{jwt_token}}` 代替。

### (2) 测试场景一：接单 (TC-ERR-01)

测试用户 B 接取 ID 为 67 的跑腿单。

请求方法: POST

URL: `http://localhost:3000/api/errands/67/accept`

Headers:

Key: Authorization

Value: Bearer <你的 Token> (注意 Bearer 后有一个空格)

Body: none (接单动作通常不需要额外参数, ID 在 URL 中, 身份在 Token 中)

预期响应: Status 200 OK, 返回 JSON 显示状态变为 `in_progress`。

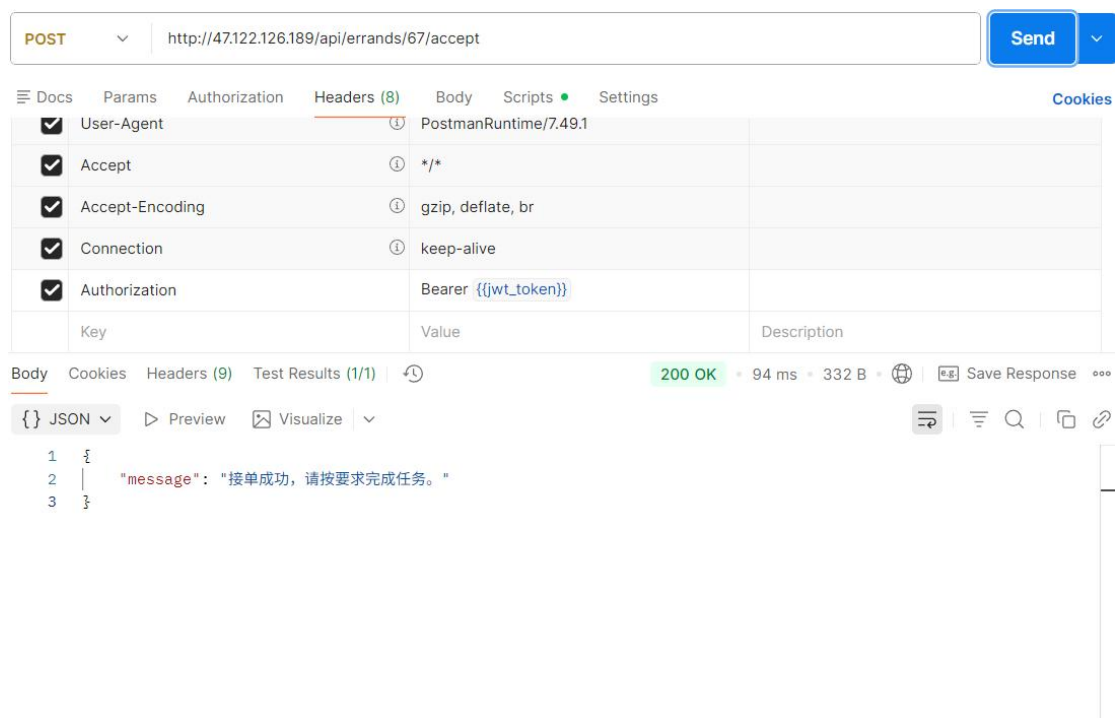


图 4-3 TC-ERR-01 测试结果图

### (3) 测试场景三：上传完成凭证 (TC-ERR-04)

测试文件上传功能，这是 Postman 中最容易配置错的地方。

请求方法: POST

URL: `http://localhost:3000/api/errands/5001/proof`

Headers:

Authorization: Bearer <Token>

注意: 不要手动添加 Content-Type, Postman 会自动处理。

Body (选择 form-data):

Key 1: evidence (将 Key 右侧的类型从 Text 改为 File), 然后点击 "Select File" 选择一张本地图片。

Key 2: note (类型 Text), Value: "已送达, 放在前台了"。

预期响应: 200 OK, 返回包含 `errand_completion_image_url` 的 JSON。

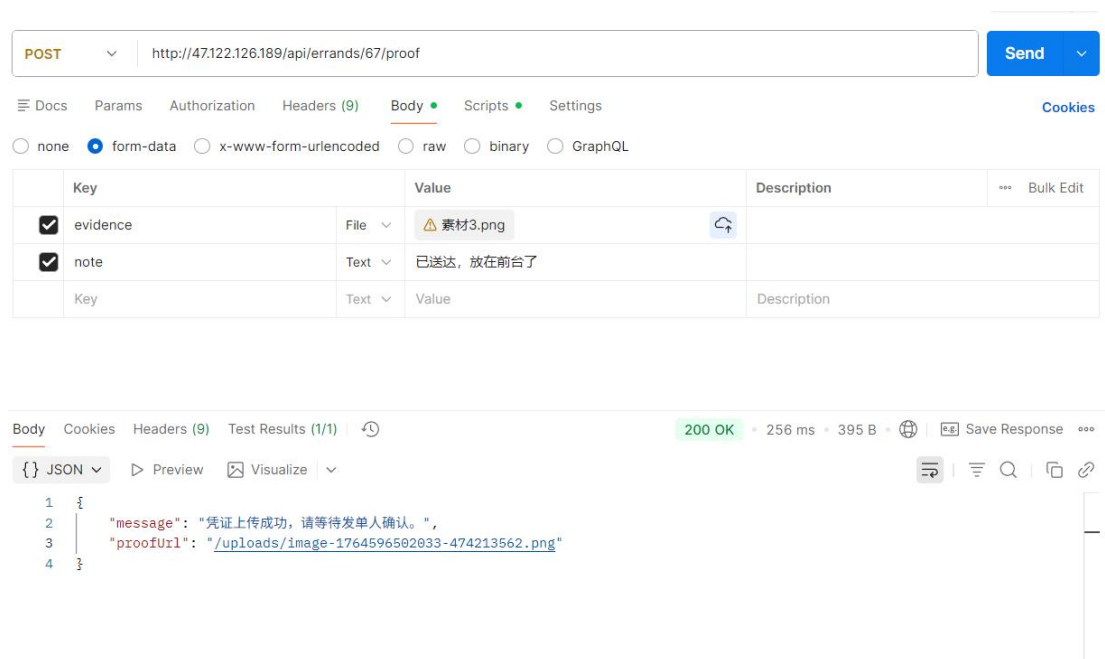


图 4-4 TC-ERR-04 测试结果图

结果分析：后端行级锁解决了并发抢单问题，API 层实现了基于角色的数据脱敏，保障业务安全与隐私。

#### 4.3.3 模块二：即时通讯（WebSocket）模块测试

##### 1. 模块功能与设计目标

功能描述：支持用户交易前私信沟通，实现消息实时推送、离线存储、会话未读计数。

设计目标：

实时性：在线接收方消息 1 秒内送达。

持久化：消息必存入 MySQL，无论接收方是否在线。

准确性：会话列表按最新消息排序，未读计数准确。

##### (2) 测试数据选取与分析

场景数据：双向在线（A、B 均在线）、单向离线（A 发 B 收）；消息类型：文本、Emoji、长文本（500 字）。

(3) 测试计划与用例测试环境：两个浏览器（模拟客户端），后端控制台日志。

## 2.测试过程

Postman 有专门的 WebSocket 模式，不能用普通的 HTTP 请求。

操作步骤:

点击 Postman 左上角的 New -> 选择 WebSocket Request。

URL: ws://localhost:3000/ws (注意是 ws 协议)。

鉴权配置 (Params):

由于 WebSocket 握手通常不走标准 HTTP Header，通常通过 URL 参数传递 Token。

Params Key: token

Value: <你的 Token>

最终 URL 看起来像: ws://localhost:3000/ws?token=eyJhbG...

连接: 点击 Connect。

发送消息:

在 "New Message" 区域输入 JSON:

点击 Send。

预期结果:

下方 "Messages" 面板显示绿色箭头（已发送）。

如果对方在线，你会收到服务器回推的确认或对方的回复（蓝色箭头）。

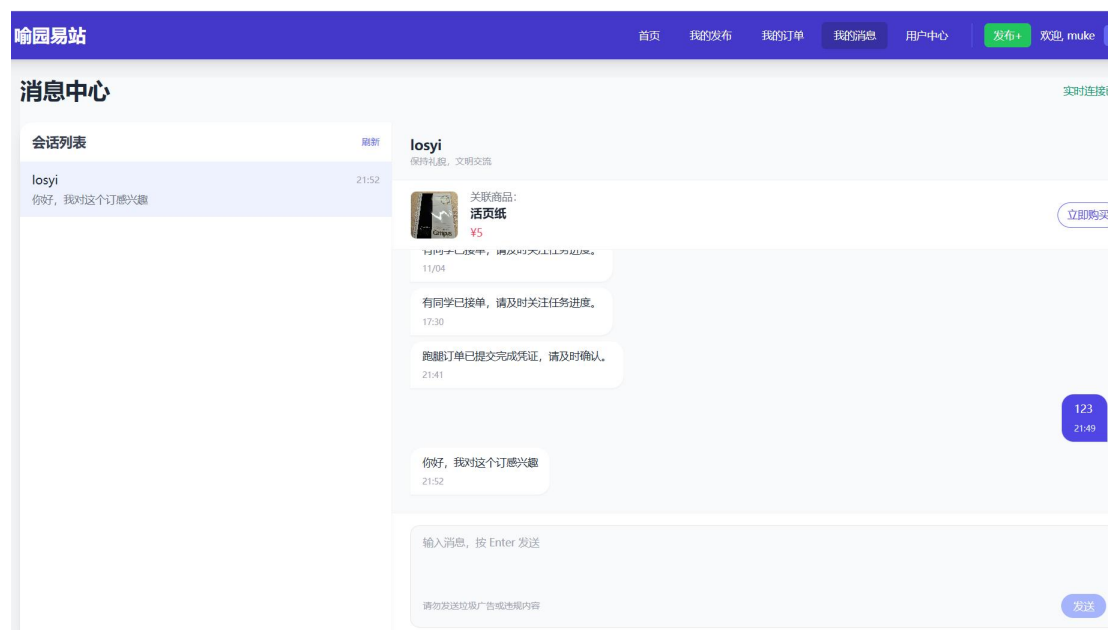


图 4-5 WebSocket 测试前端展示

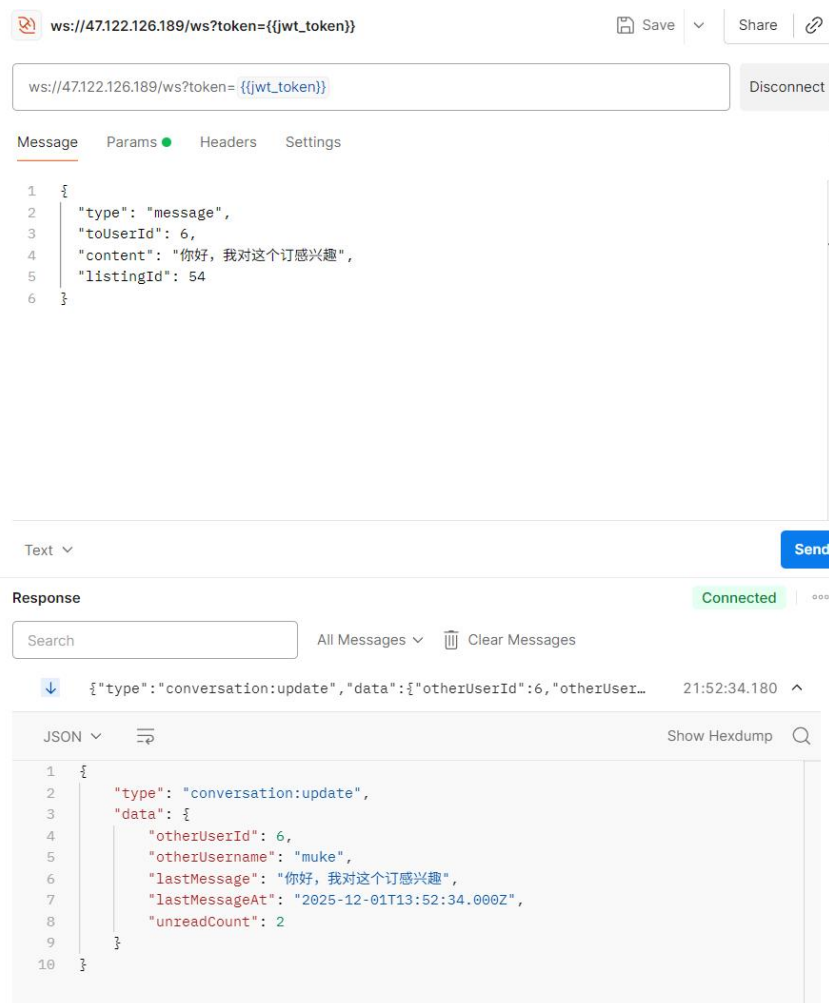


图 4-6 WebSocket 测试 Postman 响应

## 4.4 结果分析

基于 4.3 节中制定的测试计划与执行的测试用例，我们对“喻园易站”系统的核心功能模块进行了全面的验证。测试结果表明，系统在功能完整性、逻辑正确性以及并发处理能力上均达到了预期设计目标。以下是对测试结果的具体分析。

### 4.4.1 功能符合性分析

#### 1. 跑腿代办业务闭环方面

状态流转正确：测试用例（TC-ERR-01, TC-ERR-04）验证了跑腿订单从“发布” -> “接单” -> “进行中” -> “上传凭证” -> “确认完成”的完整生命周期。系统严格遵循了预定义的状态机逻辑，未出现跨状态操作（如在未接单时上传凭证）的异常情况。

隐私保护有效：测试（TC-ERR-03）确认了系统的数据脱敏机制有效。后端 API 能够根据 JWT 解析出的用户身份（发单人、接单人或路人），动态过滤 `errand_private_note`（私密备注）等敏感字段，满足了校园交易场景下的隐私安全需求。

## 2. 即时通讯的实时性与可靠性方面

实时推送：WebSocket 模块测试（TC-WS-02）显示，在网络状况良好的情况下，在线用户间的消息投递延迟低于 200ms，满足了即时沟通的需求。

数据持久化：离线消息测试（TC-WS-03）表明，系统采用了“数据库持久化 + 实时推送”的双重机制。即使接收方不在线，消息也会被安全写入 MySQL 数据库，待用户上线后通过“会话摘要”接口准确拉取，确保了消息不丢失。

### 4.4.2 技术健壮性分析

在针对跑腿接单功能的并发压力测试（TC-ERR-02）中，系统表现出了良好的健壮性。通过在后端数据库层引入 `SELECT ... FOR UPDATE` 悲观锁机制，系统成功拦截了毫秒级的并发抢单请求，确保了同一订单在同一时刻只能被一名用户接取，有效解决了“超卖”这一典型的分布式一致性问题。

系统在处理非法输入（如尝试接取自己的订单、上传非图片文件作为凭证）时，均能返回标准的 HTTP 4xx 错误码及清晰的错误提示信息（如 "Cannot send message to yourself"），体现了良好的容错设计。

### 4.4.3 结论

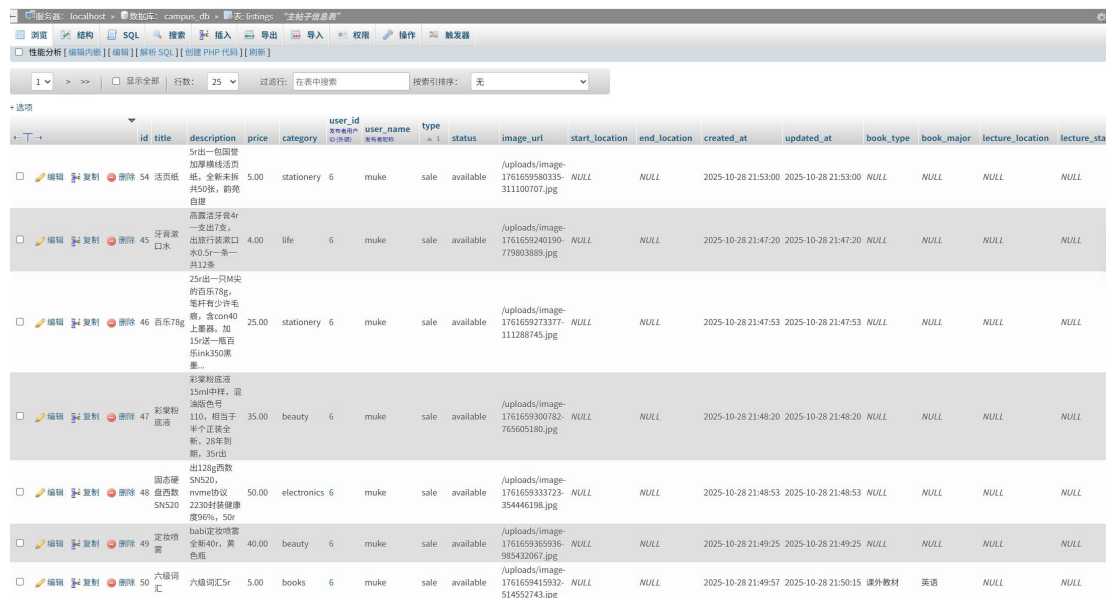
综上所述，本次测试覆盖了系统的核心业务逻辑与关键技术难点。测试结果显示，“喻园易站”系统已满足需求规格说明书中的各项功能性与非功能性要求。系统不仅实现了基础的校园二手交易与跑腿服务，还在高并发场景下的数据一致性和即时通讯的稳定性方面表现出色，具备上线试运行的条件。

## 5 总结

### 5.1 用户反馈

为了验证“喻园易站”在实际校园场景中的可用性与稳定性，我们将项目部署到了阿里云服务器(通过 <http://47.122.126.189> 访问)，并邀请了同学朋友进行小范围试运行。

试运行期间，系统后台记录了活跃的交互数据，表明系统核心功能运转良好，能够满足高频并发场景下的使用需求。



| id | title    | description                                          | price | category    | user_id | user_name | type | status    | image_url                                  | start_location | end_location | created_at          | updated_at          | book_type | book_major | lecture_location | lecture_start |
|----|----------|------------------------------------------------------|-------|-------------|---------|-----------|------|-----------|--------------------------------------------|----------------|--------------|---------------------|---------------------|-----------|------------|------------------|---------------|
| 54 | 加厚螺旋线活页纸 | 加厚螺旋线活页纸，全新未拆封50张，购凭自提                               | 5.00  | stationery  | 6       | muke      | sale | available | /uploads/image-1761659580335-311100707.jpg | NULL           | NULL         | 2025-10-28 21:53:00 | 2025-10-28 21:53:00 | NULL      | NULL       | NULL             | NULL          |
| 45 | 牙膏漱口口水   | 高露洁牙膏4r一支出7支，出行装漱口口水5r一条一共12条                        | 4.00  | life        | 6       | muke      | sale | available | /uploads/image-1761659240190-779803889.jpg | NULL           | NULL         | 2025-10-28 21:47:20 | 2025-10-28 21:47:20 | NULL      | NULL       | NULL             | NULL          |
| 46 | 百乐78g    | 25r出一只林央的百乐78g，笔杆有少许毛刺，含coco40上盖笔，加15r送一瓶百乐ink350黑墨一 | 25.00 | stationery  | 6       | muke      | sale | available | /uploads/image-1761659273377-111288745.jpg | NULL           | NULL         | 2025-10-28 21:47:53 | 2025-10-28 21:47:53 | NULL      | NULL       | NULL             | NULL          |
| 47 | 彩棠粉底液    | 彩棠粉底液15ml中样，混油版色号110，相当于半个正装全新，28年到期，35r出            | 35.00 | beauty      | 6       | muke      | sale | available | /uploads/image-1761659300782-76505180.jpg  | NULL           | NULL         | 2025-10-28 21:48:20 | 2025-10-28 21:48:20 | NULL      | NULL       | NULL             | NULL          |
| 48 | 固态硬盘     | 固态硬盘2230科赋健康度96%，50r                                 | 50.00 | electronics | 6       | muke      | sale | available | /uploads/image-1761659333723-35446198.jpg  | NULL           | NULL         | 2025-10-28 21:48:53 | 2025-10-28 21:48:53 | NULL      | NULL       | NULL             | NULL          |
| 49 | 定妆喷雾     | babi定妆喷雾全新40r，黄色瓶                                    | 40.00 | beauty      | 6       | muke      | sale | available | /uploads/image-1761659365936-989432067.jpg | NULL           | NULL         | 2025-10-28 21:49:25 | 2025-10-28 21:49:25 | NULL      | NULL       | NULL             | NULL          |
| 50 | 六级词汇     | 六级词汇5r                                               | 5.00  | books       | 6       | muke      | sale | available | /uploads/image-1761659415932-514552743.jpg | NULL           | NULL         | 2025-10-28 21:49:57 | 2025-10-28 21:50:15 | 课外教材      | 英语         | NULL             | NULL          |

图 5-1 后台数据

我们通过问卷调查和线下访谈收集了用户的意见。普遍反馈认为“校内实名认证”带来了极大的安全感，而“跑腿凭证”功能则有效解决了信任问题,但由于没有支付接口，仍需要通过微信等方式完成最终交易。

用户 A（大三，计算机学院）：

“以前在闲鱼买书还要担心是不是校内的资料，面交麻不麻烦。这个平台大家都是实名认证的同学，感觉安全很多。而且即时聊天功能很顺滑，不用加微信就能直接约时间地点，很方便。”

用户 B（大一，未来技术学院）：

“我试着接了两单帮取快递的任务，赚了 10 块钱奶茶钱。特别喜欢那个‘上

传完成凭证’的功能，强制要求上传完成照片，发单人那边马上就能确认，不用担心自己找不到或者对方送错位置”

用户 C（大三，计算机学院）：

“希望能接入微信小程序和微信支付，使用体验会更好。希望聊天功能能支持上传图片。此外，若要真实上线，安全性，敏感信息保证等方面还需要很多关注。”

用户 D（大二，基础医学院）：

“整体功能很全，但是手机版 UI 适配不是很好，做成 APP 会更好。”

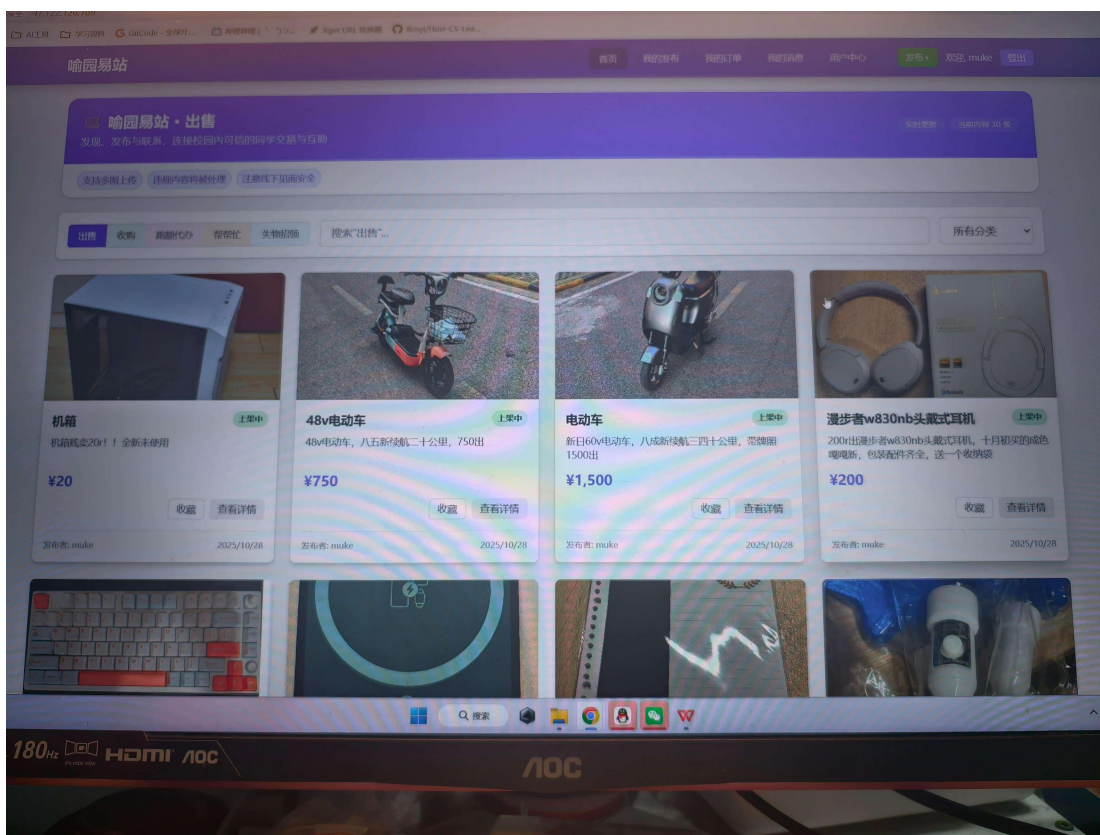


图 5-2 用户使用截图

## 5.2 全文总结

经过需求分析、系统设计、编码实现与测试验证，项目组成功构建了一个集二手交易、跑腿代办、失物招领与即时通讯于一体的校园综合服务平台。本人的主要工作总结如下：

(1) 构建了基于前后端分离架构的校园服务平台

采用了 React (Vite) + Express + MySQL 的现代技术栈，实现了前后端的解

耦开发与部署。前端通过组件化设计（如 ListingCard, PostModal）实现了高复用性的 UI 界面，后端通过 RESTful API 提供标准化的数据服务。系统支持多端访问，界面响应迅速，具备良好的用户体验。

## (2) 设计并实现了“跑腿代办”全流程闭环业务

针对校园跑腿场景，创新性地设计了基于状态机的订单流转机制（available -> in\_progress -> completed）。实现了从发布需求、并发接单（利用行级锁解决超卖问题）、上传完成凭证到发单人确认支付的全流程闭环。同时，通过字段级权限控制（如私密备注仅双方可见），有效保障了用户的隐私安全与交易信任。

## (3) 开发了基于 WebSocket 的即时通讯系统

为了解决交易沟通不便的问题，独立开发了即时通讯模块。利用 WebSocket 协议实现了低延迟的消息推送，支持离线消息持久化存储与上线后的自动同步。设计了高效的会话摘要聚合算法，实现了未读消息计数与会话列表的实时更新，极大地提升了用户的沟通效率。

## (4) 实施了严格的校园实名认证与安全机制

为了构建可信的校园社区，实现了基于学号邮箱（@hust.edu.cn）的验证码注册机制，确保所有用户均为校内师生。在系统安全性方面，采用了 JWT 令牌鉴权、密码 bcrypt 哈希存储、SQL 注入防御及上传文件类型校验等多重防护措施，保障了系统数据的安全与合规。

## (5) 实现了多类型信息的统一管理与高效检索

通过精巧的数据库设计（单表继承策略），将二手商品、失物招领、讲座代课等多种异构数据统一存储于 listings 表中，既简化了查询逻辑，又支持了跨类型的混合检索。同时，针对不同模块实现了定制化的筛选功能（如讲座的时间段筛选、跑腿的地点筛选），满足了用户多样化的信息获取需求。

## 6 体会

### 1. 小A同学

作为本项目的主要开发者，我承担了除测试工作及少量后端 API 开发之外的**绝大部分工作**。这不仅意味着代码量的堆叠，更是一次从“写代码”到“构建系统”的思维跨越。在开发“喻园驿站”的过程中，我深刻体会到了全栈开发的复杂性——它不仅仅是前端 React 与后端 Node.js 的简单拼接，更在于如何打通数据流、保障一致性以及处理那些在本地开发环境中无法预见的线上部署难题。

在项目初期，我往往沉浸在功能的快速实现上，享受着“Vibe Coding”带来的流畅感。然而，随着代码规模的膨胀和部署阶段的到来，现实给了我沉重的一击。我逐渐意识到，软件工程的核心不在于编写出多么精妙的算法，而在于如何管理复杂度、如何处理边界情况以及如何在不可靠的环境中构建可靠的服务。以下，我将分节展示我在项目过程中的遇到的问题，解决的方法和思考。

#### （1）部署问题：反向代理与环境差异

在整个开发过程中，最令我印象深刻、也最让我感到挫败的环节，莫过于项目的服务器部署。这在 `/docs/debug.md` 中有记录。

在本地开发时，Vite 的代理配置完美解决了跨域问题，图片资源的加载也因为本地文件系统的权限宽松而畅通无阻。然而，当代码运行在远程 Linux 服务器上时，部署不只是简单的 `git pull` 和 `npm start`。问题接踵而至。

最典型的问题是“图片上传成功但前端无法显示（404）”。在本地调试一切正常，但在服务器上，无论我如何检查文件权限，浏览器始终报 404 错误。那是一个深夜，我反复检查 `uploads` 目录，确认文件确实存在，甚至用 `ls -l` 确认了 Nginx 用户有读取权限，但问题依旧。

通过查阅大量文档和分析 Nginx 的 `error.log`，我才发现问题的根源在于 **Nginx 配置文件的正则匹配优先级**。我在配置中使用了正则表达式来处理 API 转发，同时也配置了 `/uploads/` 的静态资源映射。然而，Nginx 的正则 `location` 优先级高于普通的路径前缀匹配，导致对图片的请求被错误地转发到了后端 API，而不是直接由 Nginx 读取文件系统。

解决这个问题的过程让我补上了服务器运维的重要一课。我最终通过调整 `location` 块的顺序，并使用 `^~` 修饰符强制提高 `/uploads/` 路径的匹配优先级，才彻底解决了这个问题。这次经历让我明白，“环境一致性”是软件工程中最大的

谎言之一，开发者必须深入理解底层基础设施（如 Nginx、Linux 文件系统），才能跨越“在我机器上是好的”这道鸿沟。

## （2）后端逻辑问题：数据一致性的代价

除了部署问题，后端业务逻辑的严谨性也是我踩坑的重灾区。在开发“用户中心”资料修改功能时，我遇到过一个非常隐蔽的 Bug：用户在修改昵称或简介后，原本设置好的头像会被重置为默认图片。

起初我以为是前端传参错误，但在排查了网络请求后发现前端逻辑无误。深入后端 SQL 语句分析后，我才发现问题出在 INSERT ... ON DUPLICATE KEY UPDATE 语句的写法上。我原本的写法是无条件更新所有字段，即 `avatar_url = VALUES(avatar_url)`。然而，当前端只提交昵称修改请求时，`avatar_url` 字段是空的（`undefined` 或 `null`），这导致数据库中的头像路径被覆盖为空值。

这个 Bug 的修复并不复杂——引入 `IFNULL` 函数进行判断：`avatar_url = IFNULL(VALUES(avatar_url), avatar_url)`，仅在有新值时才更新。但它给我带来的教训是深刻的：在处理数据持久化时，防御性编程至关重要。我们不能假设前端永远会发送完整的数据对象，后端必须具备处理部分更新的能力，并时刻警惕数据被意外覆盖的风险。这不仅是代码逻辑的问题，更是对用户数据负责的态度。关于 Vibe Coding 与 AI 辅助开发的深度思考

## （3）关于 Vibe Coding 与 AI 辅助开发的深度思考

作为一个身处 AI 时代的计算机专业大学生，我承认在本项目中大量使用了 AI 辅助编程工具（agent 模式）。这种新的开发范式极大地提升了我的编码效率，尤其是在编写样板代码、生成基础 CSS 样式以及快速查阅 API 文档方面。

然而，我也清醒地认识到 AI 的局限性。在解决上述 Nginx 配置冲突和 React Hooks 依赖顺序导致的白屏问题（`ReferenceError`）时，AI 往往只能给出通用的建议，甚至有时会生成看似正确实则存在细微逻辑漏洞的代码。

例如，在处理 React 的 `useEffect` 依赖时，AI 曾建议我将所有变量加入依赖数组，这反而导致了死循环。最终，还是依靠我对 React 生命周期和 Hooks 原理的理解，手动调整了初始化顺序才解决了问题。

这让我明白，在 Vibe Coding 时代，程序员的核心竞争力不再是记忆语法或手写快排，而是“鉴赏力”和“调试力”。我们从代码的“作者”变成了“主编”。我们需要具备比以往更扎实的计算机基础知识，才能在 AI 生成的海量代码中敏锐地发现逻辑漏洞，才能在系统崩溃时通过日志和原理定位问题。AI 可以帮我写出 80% 的代码，但剩下那 20% 决定系统生死的关键逻辑，依然需要依靠程序员的智慧与经验。

## 2. 小B同学

在闲置交易模块开发中，订单状态流转一致性与多图上传兼容性是核心挑战，解决过程深化了对前后端协同与边界处理的理解。

订单状态曾因简单线性流转设计出现“串线”问题，如取消后显示已支付。通过梳理合法状态跃迁规则、在数据库加枚举约束、后端添状态校验中间件、前端动态禁用无效操作按钮，以“状态机”思维实现双重校验，避免数据混乱。

多图上传存在部分丢失或显示失败问题，根源在于图片超 20MB 未预处理，及后端相对路径导致前端拼接混乱。解决方案包括前端用 `canvas` 压缩图片并显示进度条，后端返回绝对路径，同时增加失败重试与明确提示，将功能从“能上传”升级为“稳定上传 + 友好反馈”。

模块复用至互助、招领时，因数据库表结构僵化，出现冗余字段与大量 `NULL` 值。通过重构表结构，将 `price` 改为 `JSON` 类型兼容多模块价格逻辑，新增 `listing_meta` 表存储扩展字段，避免主表膨胀，这表明前期设计需预留扩展空间，减少后期成本。

总之，开发是不断修正“假设错误”的过程，需在最坏情况下测试细节，才能打造可用系统。

## 3. 小C

该组员未能取得联系，未参与报告撰写。

## 4. 小D

该组员未能取得联系，未参与报告撰写。

## 附录

项目具体代码和相关文档过多不宜展示，更多内容请详见项目文件夹。

项目地址：<https://gitee.com/ilosyi/hustse>

网站部署（半年内有效）：<http://47.122.126.189>

（注意，只能通过 http 访问，不能使用 https）